Part III

Conditioning and Stability

Lecture 12. Conditioning and Condition Numbers

In this third part of the book we turn to a systematic discussion of two fundamental issues of numerical analysis that until now we have only skirted. *Conditioning* pertains to the perturbation behavior of a mathematical problem. *Stability* pertains to the perturbation behavior of an algorithm used to solve that problem on a computer.

Condition of a Problem

In the abstract, we can view a *problem* as a function $f : X \to Y$ from a normed vector space X of data to a normed vector space Y of solutions. This function f is usually nonlinear (even in linear algebra), but most of the time it is at least continuous.

Typically we shall be concerned with the behavior of a problem f at a particular data point $x \in X$ (the behavior may vary greatly from one point to another). The combination of a problem f with prescribed data x might be called a *problem instance*, but it is more usual, though occasionally confusing, to use the term *problem* for this notion too.

A *well-conditioned* problem (instance) is one with the property that all small perturbations of x lead to only small changes in $f(x)$. An *ill-conditioned* problem is one with the property that some small perturbation of x leads to a large change in $f(x)$.

The meaning of "small" and "large" in these statements depends on the application. In particular, sometimes it is most appropriate to measure perturbations on an absolute scale, and sometimes it is most appropriate to measure them relative to the norm of the object being perturbed.

Absolute Condition Number

Let δx denote a small perturbation of x, and write $\delta f = f(x + \delta x) - f(x)$. The *absolute condition number* $\hat{\kappa} = \hat{\kappa}(x)$ of the problem f at x is defined as

$$\hat{\kappa} = \lim_{\delta \to 0} \sup_{\|\delta x\| \le \delta} \frac{\|\delta f\|}{\|\delta x\|}. \tag{12.1}$$

For most problems, the limit of the supremum in this formula can be interpreted as a supremum over all infinitesimal perturbations δx, and in the interest of readability, we shall generally write the formula simply as

$$\hat{\kappa} = \sup_{\delta x} \frac{\|\delta f\|}{\|\delta x\|}, \tag{12.2}$$

with the understanding that δx and δf are infinitesimal.

If f is differentiable, we can evaluate the condition number by means of the derivative of f. Let $J(x)$ be the matrix whose i, j entry is the partial derivative $\partial f_i / \partial x_j$ evaluated at x, known as the *Jacobian* of f at x. The definition of the derivative gives us, to first order, $\delta f \approx J(x)\delta x$, with equality in the limit $\|\delta x\| \to 0$. The absolute condition number becomes

$$\hat{\kappa} = \|J(x)\|, \tag{12.3}$$

where $\|J(x)\|$ represents the norm of $J(x)$ induced by the norms on X and Y.

Relative Condition Number

When we are concerned with relative changes, we need the notion of relative condition. The *relative condition number* $\kappa = \kappa(x)$ is defined by

$$\kappa = \lim_{\delta \to 0} \sup_{\|\delta x\| \le \delta} \left(\frac{\|\delta f\|}{\|f(x)\|} \Big/ \frac{\|\delta x\|}{\|x\|} \right), \tag{12.4}$$

or, again assuming δx and δf are infinitesimal,

$$\kappa = \sup_{\delta x} \left(\frac{\|\delta f\|}{\|f(x)\|} \Big/ \frac{\|\delta x\|}{\|x\|} \right). \tag{12.5}$$

If f is differentiable, we can express this quantity in terms of the Jacobian:

$$\kappa = \frac{\|J(x)\|}{\|f(x)\|/\|x\|}. \tag{12.6}$$

Both absolute and relative condition numbers have their uses, but the latter are more important in numerical analysis. This is ultimately because the floating point arithmetic used by computers introduces relative errors rather than absolute ones; see the next lecture. A problem is *well-conditioned* if κ is small (e.g., 1, 10, 10^2), and *ill-conditioned* if κ is large (e.g., 10^6, 10^{16}).

Examples

Example 12.1. Consider the trivial problem of obtaining the scalar $x/2$ from $x \in \mathbb{C}$. The Jacobian of the function $f : x \mapsto x/2$ is just the derivative $J = f' = 1/2$, so by (12.6),

$$\kappa = \frac{\|J\|}{\|f(x)\|/\|x\|} = \frac{1/2}{(x/2)/x} = 1.$$

This problem is well-conditioned by any standard. □

Example 12.2. Consider the problem of computing $\sqrt{x}$ for $x > 0$. The Jacobian of $f : x \mapsto \sqrt{x}$ is the derivative $J = f' = 1/(2\sqrt{x})$, so we have

$$\kappa = \frac{\|J\|}{\|f(x)\|/\|x\|} = \frac{1/(2\sqrt{x})}{\sqrt{x}/x} = \frac{1}{2}.$$

Again, this is a well-conditioned problem. □

Example 12.3. Consider the problem of obtaining the scalar $f(x) = x_1 - x_2$ from the vector $x = (x_1, x_2)^* \in \mathbb{C}^2$. For simplicity, we use the ∞-norm on the data space $\mathbb{C}^2$. The Jacobian of f is

$$J = \begin{bmatrix} \dfrac{\partial f}{\partial x_1} & \dfrac{\partial f}{\partial x_2} \end{bmatrix} = \begin{bmatrix} 1 & -1 \end{bmatrix},$$

with $\|J\|_\infty = 2$. The condition number is thus

$$\kappa = \frac{\|J\|_\infty}{\|f(x)\|/\|x\|} = \frac{2}{|x_1 - x_2|/\max\{|x_1|, |x_2|\}}.$$

This quantity is large if $|x_1 - x_2| \approx 0$, so the problem is ill-conditioned when $x_1 \approx x_2$, matching our intuition of the hazards of "cancellation error." □

Example 12.4. Consider the computation of $f(x) = \tan x$ for x near 10^{100}. In this problem, minuscule relative perturbations in x can result in arbitrarily large changes in $\tan x$. The result: $\tan(10^{100})$ is effectively uncomputable on most computers. The same minuscule perturbations result in arbitrary changes in the derivative of $\tan x$, so there is little point in trying to calculate the Jacobian other than to observe that it is not small. For a story whose punch line depends on precisely this ill-conditioning of $\tan(10^{100})$, see "Lucky Numbers" in Richard Feynman's *Surely You're Joking, Mr. Feynman.* □

Example 12.5. The determination of the roots of a polynomial, given the coefficients, is a classic example of an ill-conditioned problem. Consider $x^2 - 2x + 1 = (x-1)^2$, with a double root at $x = 1$. A small perturbation in the coefficients may lead to a larger change in the roots; for example, $x^2 - 2x + 0.9999 = (x - 0.99)(x - 1.01)$. In fact, the roots can change in proportion to the square root of the change in the coefficients, so in this case the Jacobian is infinite (the problem is not differentiable), and $\kappa = \infty$.

Polynomial rootfinding is typically ill-conditioned even in cases that do not involve multiple roots. If the ith coefficient a_i of a polynomial $p(x)$ is perturbed by an infinitesimal quantity δa_i, the perturbation of the jth root x_j is $\delta x_j = -(\delta a_i) x_j^i / p'(x_j)$, where p' denotes the derivative of p. The condition number of x_j with respect to perturbations of the single coefficient a_i is therefore

$$\kappa \;=\; \frac{|\delta x_j|}{|x_j|} \Bigg/ \frac{|\delta a_i|}{|a_i|} \;=\; \frac{|a_i x_j^{i-1}|}{|p'(x_j)|}. \tag{12.7}$$

This number is often very large. Consider the "Wilkinson polynomial"

$$p(x) \;=\; \prod_{i=1}^{20} (x - i) \;=\; a_0 + a_1 x + \cdots + a_{19} x^{19} + x^{20}. \tag{12.8}$$

The most sensitive root of this polynomial is $x = 15$, and it is most sensitive to changes in the coefficient $a_{15} \approx 1.67 \times 10^9$. The condition number is

$$\kappa \;\approx\; \frac{1.67 \times 10^9 \cdot 15^{14}}{5!\,14!} \;\approx\; 5.1 \times 10^{13}.$$

Figure 12.1 illustrates the ill-conditioning graphically. □

Example 12.6. The problem of computing the eigenvalues of a nonsymmetric matrix is also often ill-conditioned. One can see this by comparing the two matrices

$$\begin{bmatrix} 1 & 1000 \\ 0 & 1 \end{bmatrix}, \qquad \begin{bmatrix} 1 & 1000 \\ 0.001 & 1 \end{bmatrix},$$

whose eigenvalues are $\{1, 1\}$ and $\{0, 2\}$, respectively. On the other hand, if a matrix A is symmetric (more generally, if it is normal), then its eigenvalues are well-conditioned. It can be shown that if λ and $\lambda + \delta\lambda$ are corresponding eigenvalues of A and $A + \delta A$, then $|\delta\lambda| \le \|\delta A\|_2$, with equality if δA is a multiple of the identity (Exercise 26.3). Thus the absolute condition number of the symmetric eigenvalue problem is $\hat{\kappa} = 1$, if perturbations are measured in the 2-norm, and the relative condition number is $\kappa = \|A\|_2 / |\lambda|$. □

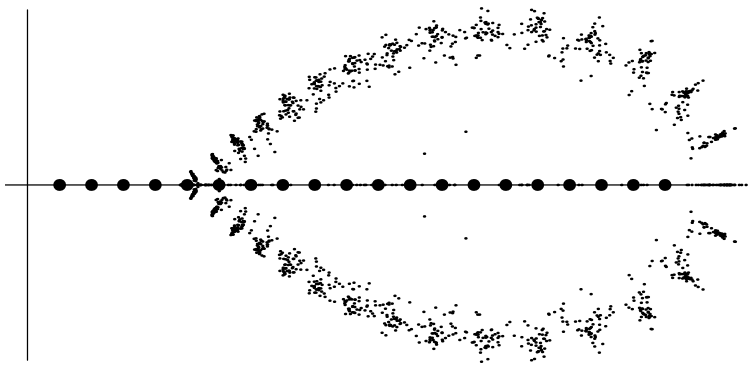


Figure 12.1. *Wilkinson's classic example of ill-conditioning. The large dots are the roots of the unperturbed polynomial* (12.8). *The small dots are the superimposed roots in the complex plane of* 100 *randomly perturbed polynomials with coefficients defined by* $\tilde{a}_k = a_k(1 + 10^{-10} r_k)$, *where* r_k *is a number from the normal distribution of mean* 0 *and variance* 1.

Condition of Matrix-Vector Multiplication

Now we come to one of the condition numbers of fundamental importance in numerical linear algebra.

Fix $A \in \mathbb{C}^{m \times n}$ and consider the problem of computing Ax from input x; that is, we are going to determine a condition number corresponding to perturbations of x but not A. Working directly from the definition of κ, with $\|\cdot\|$ denoting an arbitrary vector norm and the corresponding induced matrix norm, we find

$$\kappa = \sup_{\delta x} \left(\frac{\|A(x+\delta x) - Ax\|}{\|Ax\|} \Big/ \frac{\|\delta x\|}{\|x\|} \right) = \sup_{\delta x} \frac{\|A\delta x\|}{\|\delta x\|} \Big/ \frac{\|Ax\|}{\|x\|},$$

that is,

$$\kappa = \|A\| \frac{\|x\|}{\|Ax\|} \tag{12.9}$$

(a special case of (12.6)). This is an exact formula for κ, dependent on both A and x.

Suppose in the above calculation that A happens to be square and nonsingular. Then we can use the fact that $\|x\|/\|Ax\| \le \|A^{-1}\|$ to loosen (12.9) to a bound independent of x:

$$\kappa \le \|A\| \, \|A^{-1}\|. \tag{12.10}$$

Or, one might write

$$\kappa = \alpha \|A\| \, \|A^{-1}\| \tag{12.11}$$

with

$$\alpha = \frac{\|x\|}{\|Ax\|} \Big/ \|A^{-1}\|. \tag{12.12}$$

For certain choices of x, we have $\alpha = 1$, and consequently $\kappa = \|A\|\|A^{-1}\|$. If $\|\cdot\| = \|\cdot\|_2$, this will occur whenever x is a multiple of a minimal right singular vector of A.

In fact, A need not have been square. If $A \in \mathbb{C}^{m\times n}$ with $m \geq n$ has full rank, equations (12.10)–(12.12) hold with A^{-1} replaced by the pseudoinverse A^+ defined in (11.11).

What about the inverse problem: given A, compute $A^{-1}b$ from input b? Mathematically, this is identical to the problem just considered, except that A has been replaced by A^{-1}. Thus we have already proved the following theorem.

Theorem 12.1. *Let $A \in \mathbb{C}^{m\times m}$ be nonsingular and consider the equation $Ax = b$. The problem of computing b, given x, has condition number*

$$\kappa \;=\; \|A\|\frac{\|x\|}{\|b\|} \;\leq\; \|A\|\|A^{-1}\| \tag{12.13}$$

with respect to perturbations of x. The problem of computing x, given b, has condition number

$$\kappa \;=\; \|A^{-1}\|\frac{\|b\|}{\|x\|} \;\leq\; \|A\|\|A^{-1}\| \tag{12.14}$$

with respect to perturbations of b. If $\|\cdot\| = \|\cdot\|_2$, then equality holds in (12.13) *if x is a multiple of a right singular vector of A corresponding to the minimal singular value σ_m, and equality holds in* (12.14) *if b is a multiple of a left singular vector of A corresponding to the maximal singular value σ_1.*

Condition Number of a Matrix

The product $\|A\|\|A^{-1}\|$ comes up so often that it has its own name: it is the *condition number of A* (relative to the norm $\|\cdot\|$), denoted by $\kappa(A)$:

$$\kappa(A) = \|A\|\|A^{-1}\|. \tag{12.15}$$

Thus, in this case the term "condition number" is attached to a matrix, not a problem. If $\kappa(A)$ is small, A is said to be *well-conditioned;* if $\kappa(A)$ is large, A is *ill-conditioned.* If A is singular, it is customary to write $\kappa(A) = \infty$.

Note that if $\|\cdot\| = \|\cdot\|_2$, then $\|A\| = \sigma_1$ and $\|A^{-1}\| = 1/\sigma_m$. Thus

$$\kappa(A) = \frac{\sigma_1}{\sigma_m} \tag{12.16}$$

in the 2-norm, and it is this formula that is generally used for computing 2-norm condition numbers of matrices. The ratio σ_1/σ_m can be interpreted as

the eccentricity of the hyperellipse that is the image of the unit sphere of $\mathbb{C}^m$ under A (Figure 4.1).

For a rectangular matrix $A \in \mathbb{C}^{m\times n}$ of full rank, $m \geq n$, the condition number is defined in terms of the pseudoinverse: $\kappa(A) = \|A\|\|A^+\|$. Since A^+ is motivated by least squares problems, this definition is most useful in the case $\|\cdot\| = \|\cdot\|_2$, where we have

$$\kappa(A) = \frac{\sigma_1}{\sigma_n}. \tag{12.17}$$

Condition of a System of Equations

In Theorem 12.1, we held A fixed and perturbed x or b. What happens if we perturb A? Specifically, let us hold b fixed and consider the behavior of the problem $A \mapsto x = A^{-1}b$ when A is perturbed by infinitesimal δA. Then x must change by infinitesimal δx, where

$$(A + \delta A)(x + \delta x) \;=\; b.$$

Using the equality $Ax = b$ and dropping the doubly infinitesimal term $(\delta A)(\delta x)$, we obtain $(\delta A)x + A(\delta x) = 0$, that is, $\delta x \;=\; -A^{-1}(\delta A)x$. This equation implies $\|\delta x\| \;\leq\; \|A^{-1}\|\,\|\delta A\|\,\|x\|$, or equivalently,

$$\frac{\|\delta x\|}{\|x\|} \Big/ \frac{\|\delta A\|}{\|A\|} \;\leq\; \|A^{-1}\|\,\|A\| \;=\; \kappa(A).$$

Equality in this bound will hold whenever δA is such that

$$\|A^{-1}(\delta A)x\| = \|A^{-1}\|\|\delta A\|\|x\|,$$

and it can be shown by the use of dual norms (Exercise 3.6) that for any A and b and norm $\|\cdot\|$, such perturbations δA exist. This leads us to the following result.

Theorem 12.2. *Let b be fixed and consider the problem of computing $x = A^{-1}b$, where A is square and nonsingular. The condition number of this problem with respect to perturbations in A is*

$$\kappa \;=\; \|A\|\,\|A^{-1}\| \;=\; \kappa(A). \tag{12.18}$$

Theorems 12.1 and 12.2 are of fundamental importance in numerical linear algebra, for they determine how accurately one can solve systems of equations. If a problem $Ax = b$ contains an ill-conditioned matrix A, one must always expect to "lose $\log_{10}\kappa(A)$ digits" in computing the solution, except under very special circumstances. We shall return to this phenomenon later, and analogous results for least squares problems will be discussed in Lecture 18.

Exercises

12.1. Suppose A is a 202×202 matrix with $\|A\|_2 = 100$ and $\|A\|_F = 101$. Give the sharpest possible lower bound on the 2-norm condition number $\kappa(A)$.

12.2. In Example 11.1 we remarked that polynomial interpolation in equispaced points is ill-conditioned. To illustrate this phenomenon, let $x_1, \ldots, x_n$ and $y_1, \ldots, y_m$ be n and m equispaced points from -1 to 1, respectively.

(a) Derive a formula for the $m \times n$ matrix A that maps an n-vector of data at $\{x_j\}$ to an m-vector of sampled values $\{p(y_j)\}$, where p is the degree $n-1$ polynomial interpolant of the data (see Example 1.1).

(b) Write a program to calculate A and plot $\|A\|_\infty$ on a semilog scale for $n = 1, 2, \ldots, 30$, $m = 2n-1$. In the continuous limit $m \to \infty$, the numbers $\|A\|_\infty$ are known as the *Lebesgue constants* for equispaced interpolation, which are asymptotic to $2^n/(e\,(n-1)\log n)$ as $n \to \infty$.

(c) For $n = 1, 2, \ldots, 30$ and $m = 2n-1$, what is the ∞-norm condition number κ of the problem of interpolating the constant function 1? Use (12.6).

(d) How close is your result for $n = 11$ to the bound implicit in Figure 11.1?

12.3. The goal of this problem is to explore some properties of random matrices. Your job is to be a laboratory scientist, performing experiments that lead to conjectures and more refined experiments. Do not try to prove anything. Do produce well-designed plots, which are worth a thousand numbers.

Define a *random matrix* to be an $m \times m$ matrix whose entries are independent samples from the real normal distribution with mean zero and standard deviation $m^{-1/2}$. (In MATLAB, `A = randn(m,m)/sqrt(m)`.) The factor $\sqrt{m}$ is introduced to make the limiting behavior clean as $m \to \infty$.

(a) What do the eigenvalues of a random matrix look like? What happens, say, if you take 100 random matrices and superimpose all their eigenvalues in a single plot? If you do this for $m = 8$, 16, 32, 64, ..., what pattern is suggested? How does the spectral radius $\rho(A)$ (Exercise 3.2) behave as $m \to \infty$?

(b) What about norms? How does the 2-norm of a random matrix behave as $m \to \infty$? Of course, we must have $\rho(A) \le \|A\|$ (Exercise 3.2). Does this inequality appear to approach an equality as $m \to \infty$?

(c) What about condition numbers—or more simply, the smallest singular value $\sigma_{\min}$? Even for fixed m this question is interesting. What proportions of random matrices in $\mathbb{R}^{m\times m}$ seem to have $\sigma_{\min} \le 2^{-1}$, 4^{-1}, 8^{-1}, ...? In other words, what does the tail of the probability distribution of smallest singular values look like? How does the scale of all this change with m?

(d) How do the answers to (a)–(c) change if we consider random triangular instead of full matrices, i.e., upper-triangular matrices whose entries are samples from the same distribution as above?

Lecture 13. Floating Point Arithmetic

It did not take long after the invention of computers for consensus to emerge on the right way to represent real numbers on a digital machine. The secret is floating point arithmetic, the hardware analogue of scientific notation. Before we can begin to study the accuracy of the algorithms of numerical linear algebra, we must examine this topic.

Limitations of Digital Representations

Since digital computers use a finite number of bits to represent a real number, they can represent only a finite subset of the real numbers (or the complex numbers, which we discuss at the end of this lecture). This limitation presents two difficulties. First, the represented numbers cannot be arbitrarily large or small. Second, there must be gaps between them.

Modern computers represent numbers sufficiently large and small that the first constraint rarely poses difficulties. For example, the widely used IEEE double precision arithmetic permits numbers as large as 1.79×10^{308} and as small as 2.23×10^{-308}, a range great enough for most of the problems considered in this book. In other words, *overflow* and *underflow* are usually not a serious hazard (but watch out if you are asked to evaluate a determinant!).

By contrast, the problem of gaps between represented numbers is a concern throughout scientific computing. For example, in IEEE double precision arithmetic, the interval $[1, 2]$ is represented by the discrete subset

$$1, \quad 1 + 2^{-52}, \quad 1 + 2 \times 2^{-52}, \quad 1 + 3 \times 2^{-52}, \quad \ldots, \quad 2. \tag{13.1}$$

The interval $[2,4]$ is represented by the same numbers multiplied by 2,

$$2,\ \ 2+2^{-51},\ \ 2+2\times 2^{-51},\ \ 2+3\times 2^{-51},\ \ \ldots,\ \ 4,$$

and in general, the interval $[2^j, 2^{j+1}]$ is represented by (13.1) times 2^j. Thus in IEEE double precision arithmetic, the gaps between adjacent numbers are in a relative sense never larger than $2^{-52} \approx 2.22 \times 10^{-16}$. This may seem negligible, and so it is for most purposes if one uses stable algorithms (see the next lecture). But it is surprising how many carelessly constructed algorithms turn out to be unstable!

Floating Point Numbers

IEEE arithmetic is an example of an arithmetic system based on a *floating point* representation of the real numbers. This is the universal practice on general purpose computers nowadays. In a floating point number system, the position of the decimal (or binary) point is stored separately from the digits, and the gaps between adjacent represented numbers scale in proportion to the size of the numbers. This is distinguished from a *fixed point* representation, where the gaps are all of the same size.

Specifically, let us consider an idealized floating point number system defined as follows. The system consists of a discrete subset $\mathbf{F}$ of the real numbers $\mathbb{R}$ determined by an integer $\beta \geq 2$ known as the *base* or *radix* (typically 2) and an integer $t \geq 1$ known as the *precision* (24 and 53 for IEEE single and double precision, respectively). The elements of $\mathbf{F}$ are the number 0 together with all numbers of the form

$$x = \pm(m/\beta^t)\beta^e, \tag{13.2}$$

where m is an integer in the range $1 \leq m \leq \beta^t$ and e is an arbitrary integer. Equivalently, we can restrict the range to $\beta^{t-1} \leq m \leq \beta^t - 1$ and thereby make the choice of m unique. The quantity $\pm(m/\beta^t)$ is then known as the *fraction* or *mantissa* of x, and e is the *exponent*.

Our floating point number system is idealized in that it ignores over- and underflow. As a result, $\mathbf{F}$ is a countably infinite set, and it is self-similar: $\mathbf{F} = \beta\mathbf{F}$.

Machine Epsilon

The resolution of $\mathbf{F}$ is traditionally summarized by a number known as *machine epsilon*. Provisionally, let us define this number by

$$\epsilon_{\text{machine}} = \tfrac{1}{2}\beta^{1-t}. \tag{13.3}$$

(We shall modify the definition after (13.7).) This number is half the distance between 1 and the next larger floating point number. In a relative sense, this

is as large as the gaps between floating point numbers get. That is, $\epsilon_{\text{machine}}$ has the following property:

$$\text{For all } x \in \mathbb{R}, \text{ there exists } x' \in \mathbf{F} \text{ such that } |x - x'| \leq \epsilon_{\text{machine}}|x|. \quad (13.4)$$

For the values of β and t common on various computers, $\epsilon_{\text{machine}}$ usually lies between 10^{-6} and 10^{-35}. In IEEE single and double precision arithmetic, $\epsilon_{\text{machine}}$ is specified to be $2^{-24} \approx 5.96 \times 10^{-8}$ and $2^{-53} \approx 1.11 \times 10^{-16}$, respectively.

Let $\text{fl} : \mathbb{R} \to \mathbf{F}$ be a function giving the closest floating point approximation to a real number, its *rounded* equivalent in the floating point system. (For our purposes, ties can be broken arbitrarily, though the treatment of ties so as to avoid statistical bias is an interesting matter in itself.) The inequality (13.4) can be stated in terms of fl:

$$\begin{gathered}\text{For all } x \in \mathbb{R}, \text{ there exists } \epsilon \text{ with } |\epsilon| \leq \epsilon_{\text{machine}} \\ \text{such that } \text{fl}(x) = x(1+\epsilon).\end{gathered} \quad (13.5)$$

That is, the difference between a real number and its closest floating point approximation is always smaller than $\epsilon_{\text{machine}}$ in relative terms.

Floating Point Arithmetic

It is not enough to represent real numbers, of course; one must compute with them. On a computer, all mathematical computations are reduced to certain elementary arithmetic operations, of which the classical set is $+$, $-$, $\times$, and $\div$. Mathematically, these symbols represent operations on $\mathbb{R}$. On a computer, they have analogues that are operations on $\mathbf{F}$. It is common practice to denote these floating point operations by $\oplus$, $\ominus$, $\otimes$, and $\oslash$.

A computer might be built on the following design principle. Let x and y be arbitrary floating point numbers, that is, $x, y \in \mathbf{F}$. Let $*$ be one of the operations $+$, $-$, $\times$, or $\div$, and let $\circledast$ be its floating point analogue. Then $x \circledast y$ must be given exactly by

$$x \circledast y = \text{fl}(x * y). \quad (13.6)$$

If this property holds, then from (13.5) and (13.6) we conclude that the computer has a simple and powerful property.

Fundamental Axiom of Floating Point Arithmetic

For all $x, y \in \mathbf{F}$, there exists ϵ with $|\epsilon| \leq \epsilon_{\text{machine}}$ such that

$$x \circledast y = (x * y)(1+\epsilon). \quad (13.7)$$

In words, *every operation of floating point arithmetic is exact up to a relative error of size at most* $\epsilon_{\text{machine}}$.

Machine Epsilon, Again

The rounding error analysis in this book is based on (13.5) and (13.7), not on the other details of floating point arithmetic described above. This means that we can be generous in allowing for hardware implementations that may not perform floating point computations as perfectly as indicated by (13.6). For such a machine, (13.5) and (13.7) may still be satisfied if $\epsilon_{\text{machine}}$ is replaced by a somewhat larger value. For example, on a computer in which intermediate quantities are truncated rather than rounded, (13.7) may hold with $\epsilon_{\text{machine}}$ replaced by $2\epsilon_{\text{machine}}$.

The simplest way to allow for such complications is to retain (13.5) and (13.7) as written, but to modify the definition of $\epsilon_{\text{machine}}$. From now on, let us assume that $\epsilon_{\text{machine}}$ is defined not by (13.3), but as the smallest number for which (13.5) and (13.7) hold. For most computers, including all those implementing IEEE arithmetic, this change in the definition of $\epsilon_{\text{machine}}$ makes no significant change in its value.

Occasionally an unexpectedly large value of $\epsilon_{\text{machine}}$ may be needed to make (13.7) hold. In late 1994 the Intel Pentium^{TM} microprocessor acquired notoriety when it was discovered that because of a bug in a table used in implementing the double precision IEEE standard, its effective precision was eleven orders of magnitude too coarse, $\epsilon_{\text{machine}} \approx 6.1 \times 10^{-5}$. (The bug was soon corrected.) In fact, there are machines for which (13.7) holds only with $\epsilon_{\text{machine}} = 1$. For example, floating point subtraction on Cray computers produced up to the mid-1990s had this property, because the operation of subtraction was implemented without a "guard digit." Such computers are not useless, but they demand a different style of error analysis from the one in this book.

Fortunately, the benefits of the axiom (13.7), and of the adoption of uniform standards of computer arithmetic, have become widely accepted by computer manufacturers in recent years, and the number of machines on the market that fail to satisfy (13.7) with a small value of $\epsilon_{\text{machine}}$ is dwindling. Indeed, IEEE arithmetic itself is rapidly becoming the standard for computers of all sizes, including, as of 1996, all IBM-compatible personal computers and all workstations manufactured by SUN, DEC, Hewlett-Packard, and IBM.

Complex Floating Point Arithmetic

Floating point complex numbers are generally represented as pairs of floating point real numbers, and the elementary operations upon them are computed by reduction to real and imaginary parts. The result is that the axiom (13.7) is valid for complex as well as real floating point numbers, except that for $\otimes$ and $\oslash$, $\epsilon_{\text{machine}}$ must be enlarged from (13.3) by factors on the order of $2^{3/2}$ and $2^{5/2}$, respectively. Once $\epsilon_{\text{machine}}$ is adjusted in this manner, rounding error analysis for complex numbers can proceed just as for real numbers.

Exercises

13.1. Between an adjacent pair of nonzero IEEE single precision real numbers, how many IEEE double precision numbers are there?

13.2. The floating point system $\mathbf{F}$ defined by (13.2) includes many integers, but not all of them.

(a) Give an exact formula for the smallest positive integer n that does not belong to $\mathbf{F}$.

(b) In particular, what are the values of n for IEEE single and double precision arithmetic?

(c) Figure out a way to verify this result for your own computer. Specifically, design and run a program that produces evidence that $n-3$, $n-2$, and $n-1$ belong to $\mathbf{F}$ but n does not. What about $n+1$, $n+2$, and $n+3$?

13.3. Consider the polynomial $p(x) = (x-2)^9 = x^9 - 18x^8 + 144x^7 - 672x^6 + 2016x^5 - 4032x^4 + 5376x^3 - 4608x^2 + 2304x - 512$.

(a) Plot $p(x)$ for $x = 1.920, 1.921, 1.922, \ldots, 2.080$, evaluating p via its coefficients 1, -18, $144, \ldots$.

(b) Produce the same plot again, now evaluating p via the expression $(x-2)^9$.

13.4. The polynomial $p(x) = x^5 - 2x^4 - 3x^3 + 3x^2 - 2x - 1$ has three real zeros. Applying Newton's method to p with initial guess $x_0 = 0$ produces a series of estimates $x_1, x_2, x_3 \ldots$ that converge rapidly to a zero $x_* \approx -0.315$.

(a) Compute $x_1, \ldots, x_6$ in floating point arithmetic with $\epsilon_{\text{machine}} \approx 10^{-16}$. How many digits do you estimate are correct in each of these numbers?

(b) Compute $x_1, \ldots, x_6$ again *exactly* with the aid of a symbolic algebra system such as MAPLE or MATHEMATICA. Each x_j is a rational number. How many digits are there in the numerator and the denominator for each j?

Lecture 14. Stability

It would be a fine thing if numerical algorithms could provide exact solutions to numerical problems. Since the problems are continuous while digital computers are discrete, however, this is generally not possible. The notion of stability is the standard way of characterizing what *is* possible—numerical analysts' idea of what it means to get the "right answer," even if it is not exact.

Algorithms

In Lecture 12, we defined a mathematical *problem* as a function $f : X \to Y$ from a vector space X of data to a vector space Y of solutions.

An *algorithm* can be viewed as another map $\tilde{f} : X \to Y$ between the same two spaces. We make this definition precise as follows. Let a problem f, a computer whose floating point system satisfies (13.7) (but not necessarily (13.6)), an algorithm for f (in the loose sense of the term), and an implementation of this algorithm in the form of a computer program be fixed. Given data $x \in X$, let this data be rounded to floating point in a matter satisfying (13.5) and then supplied as input to the computer program. Now, run the program. The result is a collection of floating point numbers that belong to the vector space Y (since the algorithm was designed to solve f). Let this computed result be called $\tilde{f}(x)$.

The situation couldn't be uglier! As a minimum, $\tilde{f}(x)$ will be affected by rounding errors. Depending on the circumstances, it may also be affected by all

kinds of other complications such as convergence tolerances or even the other jobs running on the computer, in cases where the assignment of computations to processors is not determined until runtime. Thus the "function" $\tilde{f}(x)$ may even take different values from one run to the next; it may be multivalued. (In fact, the problem f should really be allowed to be multivalued too; this permits handling of cases where a nonunique solution is acceptable, e.g., either of the two square roots of a complex number.) Yet despite all these complications, we shall find that we can make surprisingly clean statements about $\tilde{f}(x)$, and hence about the accuracy of the algorithms of numerical linear algebra, based only on the fundamental axioms (13.5) and (13.7).

The tilde ($\tilde{}$) notation is very convenient. Just as $\tilde{f}$ is the computed analogue of f, other computed quantities in this book will frequently be marked by tildes. For example, the computed solution to a system of equations $Ax = b$ may be denoted by $\tilde{x}$.

Accuracy

Except in trivial cases, $\tilde{f}$ cannot be continuous. Nevertheless, a good algorithm should approximate the associated problem f. To make this idea quantitative, we may consider the *absolute error* of a computation, $\|\tilde{f}(x) - f(x)\|$, or the *relative error*,

$$\frac{\|\tilde{f}(x) - f(x)\|}{\|f(x)\|}. \tag{14.1}$$

In this book we mainly utilize relative quantities, and thus (14.1) will be our standard error measure.

If $\tilde{f}$ is a good algorithm, one might naturally expect the relative error to be small, of order $\epsilon_{\text{machine}}$. One might say that an algorithm $\tilde{f}$ for a problem f is *accurate* if for each $x \in X$,

$$\frac{\|\tilde{f}(x) - f(x)\|}{\|f(x)\|} = O(\epsilon_{\text{machine}}). \tag{14.2}$$

Loosely speaking, the symbol $O(\epsilon_{\text{machine}})$ in (14.3) means "on the order of machine epsilon." However, $O(\epsilon_{\text{machine}})$ also has a precise meaning, which we shall discuss in a moment. That discussion will also clarify how a formula like (14.2) is to be interpreted if the denominator is zero.

Stability

If the problem f is ill-conditioned, however, the goal of accuracy as defined by (14.2) is unreasonably ambitious. Rounding of the input data is unavoidable on a digital computer, and even if all the subsequent computations could be carried out perfectly, this perturbation alone might lead to a significant change in the result. Instead of aiming for accuracy in all cases, the most it

is appropriate to aim for in general is *stability*. We say that an algorithm $\tilde{f}$ for a problem f is *stable* if for each $x \in X$,

$$\frac{\|\tilde{f}(x) - f(\tilde{x})\|}{\|f(\tilde{x})\|} = O(\epsilon_{\text{machine}}) \tag{14.3}$$

for some $\tilde{x}$ with

$$\frac{\|\tilde{x} - x\|}{\|x\|} = O(\epsilon_{\text{machine}}). \tag{14.4}$$

In words,

A stable algorithm gives nearly the right answer
to nearly the right question.

The motivation for this definition will become clear in the next lecture and in applications throughout the remainder of this book.

We caution the reader that whereas the definitions of stability given here are useful in many parts of numerical linear algebra, the condition $O(\epsilon_{\text{machine}})$ is probably too strict to be appropriate for all numerical problems in other areas such as differential equations.

Backward Stability

Many algorithms of numerical linear algebra satisfy a condition that is both stronger and simpler than stability. We say that an algorithm $\tilde{f}$ for a problem f is *backward stable* if for each $x \in X$,

$$\tilde{f}(x) = f(\tilde{x}) \quad \text{for some } \tilde{x} \text{ with } \frac{\|\tilde{x} - x\|}{\|x\|} = O(\epsilon_{\text{machine}}). \tag{14.5}$$

This is a tightening of the definition of stability in that the $O(\epsilon_{\text{machine}})$ in (14.3) has been replaced by zero. In words,

A backward stable algorithm gives exactly the right answer
to nearly the right question.

Examples are given in the next lecture.

The Meaning of $O(\epsilon_{\text{machine}})$

We now explain the precise meaning of "$O(\epsilon_{\text{machine}})$" in (14.2)–(14.5).

The notation

$$\varphi(t) = O(\psi(t)) \tag{14.6}$$

is a standard one in mathematics, with a precise definition. This equation asserts that there exists some positive constant C such that, for all t sufficiently close to an understood limit (e.g., $t \to 0$ or $t \to \infty$),

$$|\varphi(t)| \le C\psi(t). \tag{14.7}$$

For example, the statement $\sin^2 t = O(t^2)$ as $t \to 0$ asserts that there exists a constant C such that, for all sufficiently small t, $|\sin^2 t| \le Ct^2$.

Also standard in mathematics are statements of the form

$$\varphi(s,t) = O(\psi(t)) \quad \text{uniformly in } s, \tag{14.8}$$

where φ is a function that depends not only on t but also on another variable s. The word "uniformly" indicates that there exists a single constant C as in (14.7) that holds for all choices of s. Thus, for example,

$$(\sin^2 t)(\sin^2 s) = O(t^2)$$

holds uniformly as $t \to 0$, but the uniformity is lost if we replace $\sin^2 s$ by s^2.

In this book, our use of the "O" symbol follows these standard definitions. Specifically, we often state results along the lines of

$$\|\text{computed quantity}\| = O(\epsilon_{\text{machine}}). \tag{14.9}$$

Here is what (14.9) means. First, "$\|$computed quantity$\|$" represents the norm of some number or collection of numbers determined by an algorithm $\tilde{f}$ for a problem f, depending both on the data $x \in X$ for f and on $\epsilon_{\text{machine}}$. An example is the relative error (14.1). Second, the implicit limit process is $\epsilon_{\text{machine}} \to 0$ (i.e., $\epsilon_{\text{machine}}$ is the variable corresponding to t in (14.8)). Third, the "O" applies uniformly for all data $x \in X$ (i.e., x is the variable corresponding to s). We shall rarely mention the uniformity with respect to $x \in X$, but it is always implicit.

In any particular machine arithmetic, the number $\epsilon_{\text{machine}}$ is a fixed quantity. In speaking of the limit $\epsilon_{\text{machine}} \to 0$, we are considering an idealization of a computer, or perhaps one should say, of a family of computers. Equation (14.9) means that if we were to run the algorithm in question on computers satisfying (13.5) and (13.7) for a sequence of values of $\epsilon_{\text{machine}}$ decreasing to zero, then $\|$computed quantity$\|$ would be guaranteed to decrease in proportion to $\epsilon_{\text{machine}}$ or faster. These ideal computers are required to satisfy (13.5) and (13.7) but nothing else.

Dependence on m and n, not A and b

It cannot hurt to discuss the meaning of $O(\epsilon_{\text{machine}})$ in (14.2)–(14.5) a bit further. The uniformity of the constant implicit in the "O" can be illustrated by the following example. Suppose we are considering an algorithm for solving a nonsingular $m \times m$ system of equations $Ax = b$ for x, and we assert that the computed result $\tilde{x}$ for this algorithm satisfies

$$\frac{\|\tilde{x} - x\|}{\|x\|} = O(\kappa(A)\,\epsilon_{\text{machine}}). \tag{14.10}$$

This assertion means that the bound

$$\frac{\|\tilde{x} - x\|}{\|x\|} \le C\kappa(A)\,\epsilon_{\text{machine}} \tag{14.11}$$

holds for a single constant C, independently of the matrix A or the right-hand side b, for all sufficiently small $\epsilon_{\text{machine}}$.

If the denominator in a formula like (14.11) is zero, its meaning is defined by the following convention. When we write (14.11), what we really mean is

$$\|\tilde{x} - x\| \le C\kappa(A)\,\epsilon_{\text{machine}}\|x\|. \tag{14.12}$$

There is no difference if $\|x\| \neq 0$, but if $\|x\| = 0$, (14.12) makes it clear that the precise meaning of (14.10) is that $\|\tilde{x} - x\| = 0$ for all sufficiently small $\epsilon_{\text{machine}}$.

Though the constant C of (14.11) or (14.12) does not depend on A or b, it does depend, in general, on the dimension m. Formally speaking, this is a consequence of our definition of a problem in Lecture 12. If the dimensions such as m or n that define a problem f change, then the vector spaces X and Y must change too, and thus we have a new problem, f'. As a practical matter, too, the effects of rounding errors on the algorithms of numerical linear algebra generally grow with m and n. However, this growth is usually slow enough that it is not serious. The dependence on m or n is typically linear, quadratic, or cubic in the worst case (the exponent depends on the choice of norm as well as the choice of algorithm), and the errors for most data are much smaller than in the worst case, thanks to statistical cancellation.

In principle, a statement like (14.9) might conceal a dimension-dependent factor such as 2^m that would make the bound useless in practice. However, there is only one place in this book where such a thing happens—in the discussion of Gaussian elimination with partial pivoting—and we shall give the reader ample warning at that point to avoid misunderstanding. As a rule, when the expression $O(\epsilon_{\text{machine}})$ is printed in this book, the chances are that in an actual calculation on an actual machine, the quantity in question will be at most 100 or perhaps 1000 times as large as $\epsilon_{\text{machine}}$.

Independence of Norm

Our definitions involving $O(\epsilon_{\text{machine}})$ have the convenient property that, provided X and Y are finite-dimensional, they are norm-independent.

Theorem 14.1. *For problems f and algorithms $\tilde{f}$ defined on finite-dimensional spaces X and Y, the properties of accuracy, stability, and backward stability all hold or fail to hold independently of the choice of norms in X and Y.*

Proof. It is well known (and easily proved) that in a finite-dimensional vector space, all norms are equivalent in the sense that if $\|\cdot\|$ and $\|\cdot\|'$ are two norms

on the same space, then there exist positive constants C_1 and C_2 such that $C_1\|x\| \le \|x\|' \le C_2\|x\|$ for all x in that space. It follows that a change of norm may affect the size of the constant C implicit in a statement involving $O(\epsilon_{\text{machine}})$, but not the existence of such a constant. □

Exercises

14.1. True or False?

(a) $\sin x = O(1)$ as $x \to \infty$.

(b) $\sin x = O(1)$ as $x \to 0$.

(c) $\log x = O(x^{1/100})$ as $x \to \infty$.

(d) $n! = O((n/e)^n)$ as $n \to \infty$.

(e) $A = O(V^{2/3})$ as $V \to \infty$, where A and V are the surface area and volume of a sphere measured in square microns and cubic miles, respectively.

(f) $\text{fl}(\pi) - \pi = O(\epsilon_{\text{machine}})$. (We do not mention that the limit is $\epsilon_{\text{machine}} \to 0$, since that is implicit for all expressions $O(\epsilon_{\text{machine}})$ in this book.)

(g) $\text{fl}(n\pi) - n\pi = O(\epsilon_{\text{machine}})$, uniformly for all integers n. (Here $n\pi$ represents the exact mathematical quantity, not the result of a floating point calculation.)

14.2. (a) Show that $(1 + O(\epsilon_{\text{machine}}))(1 + O(\epsilon_{\text{machine}})) = 1 + O(\epsilon_{\text{machine}})$. The precise meaning of this statement is that if f is a function satisfying $f(\epsilon_{\text{machine}}) = (1 + O(\epsilon_{\text{machine}}))(1 + O(\epsilon_{\text{machine}}))$ as $\epsilon_{\text{machine}} \to 0$, then f also satisfies $f(\epsilon_{\text{machine}}) = 1 + O(\epsilon_{\text{machine}})$ as $\epsilon_{\text{machine}} \to 0$.

(b) Show that $(1 + O(\epsilon_{\text{machine}}))^{-1} = 1 + O(\epsilon_{\text{machine}})$.

Lecture 15. More on Stability

We continue the discussion of stability by considering examples of stable and unstable algorithms. Then we discuss a fundamental idea linking conditioning and stability, whose power has been proved in innumerable applications since the 1950s: backward error analysis.

Stability of Floating Point Arithmetic

The four simplest computational problems are $+$, $-$, $\times$, and $\div$. There is not much to say about choice of algorithms! Of course, we shall normally use the floating point operations $\oplus$, $\ominus$, $\otimes$, and $\oslash$ provided with the computer. As it happens, the axioms (13.5) and (13.7) imply that these four canonical examples of algorithms are all backward stable.

Let us show this for subtraction, since this is the elementary operation one might expect to be at greatest risk of instability. As in Example 12.3, the data space X is the set of 2-vectors, $\mathbb{C}^2$, and the solution space Y is the set of scalars, $\mathbb{C}$. By Theorem 14.1, we need not specify the norms in these spaces.

For data $x = (x_1, x_2)^* \in X$, the problem of subtraction corresponds to the function $f(x_1, x_2) = x_1 - x_2$, and the algorithm we are considering can be written

$$\tilde{f}(x_1, x_2) = \mathrm{fl}(x_1) \ominus \mathrm{fl}(x_2).$$

This equation means that we first round x_1 and x_2 to floating point values, then apply the operation $\ominus$. Now by (13.5), we have

$$\mathrm{fl}(x_1) = x_1(1 + \epsilon_1), \qquad \mathrm{fl}(x_2) = x_2(1 + \epsilon_2)$$

for some $|\epsilon_1|, |\epsilon_2| \le \epsilon_{\text{machine}}$. By (13.7), we have

$$\text{fl}(x_1) \ominus \text{fl}(x_2) \;=\; (\text{fl}(x_1) - \text{fl}(x_2))(1+\epsilon_3)$$

for some $|\epsilon_3| \le \epsilon_{\text{machine}}$. Combining these equations gives

$$\begin{aligned} \text{fl}(x_1) \ominus \text{fl}(x_2) &= [x_1(1+\epsilon_1) - x_2(1+\epsilon_2)](1+\epsilon_3) \\ &= x_1(1+\epsilon_1)(1+\epsilon_3) - x_2(1+\epsilon_2)(1+\epsilon_3) \\ &= x_1(1+\epsilon_4) - x_2(1+\epsilon_5) \end{aligned}$$

for some $|\epsilon_4|, |\epsilon_5| \le 2\epsilon_{\text{machine}} + O(\epsilon_{\text{machine}}^2)$ (see Exercise 14.2). In other words, the computed result $\tilde{f}(x) = \text{fl}(x_1) \ominus \text{fl}(x_2)$ is exactly equal to the difference $\tilde{x}_1 - \tilde{x}_2$, where $\tilde{x}_1$ and $\tilde{x}_2$ satisfy

$$\frac{|\tilde{x}_1 - x_1|}{|x_1|} = O(\epsilon_{\text{machine}}), \qquad \frac{|\tilde{x}_2 - x_2|}{|x_2|} = O(\epsilon_{\text{machine}}),$$

and any $C > 2$ will suffice for the constants implicit in the "O" symbols. For any choice of norm $\|\cdot\|$ in the space $\mathbb{C}^2$, this implies (14.5).

Further Examples

Example 15.1. Inner Product. Suppose we are given vectors $x, y \in \mathbb{C}^m$ and wish to compute the inner product $\alpha = x^* y$. The obvious algorithm is to compute the pairwise products $\overline{x}_i y_i$ with $\otimes$ and add them with $\oplus$ to obtain a computed result $\tilde{\alpha}$. It can be shown that this algorithm is backward stable; this is done implicitly in Lecture 17. □

Example 15.2. Outer Product. On the other hand, suppose we wish to compute the rank-one outer product $A = xy^*$ for vectors $x \in \mathbb{C}^m$, $y \in \mathbb{C}^n$. The obvious algorithm is to compute the mn products $x_i \overline{y}_j$ with $\otimes$ and collect them into a matrix $\tilde{A}$. This algorithm is stable, but it is not backward stable. The explanation is that the matrix $\tilde{A}$ will be most unlikely to have rank exactly 1, and thus it cannot generally be written in the form $(x+\delta x)(y+\delta y)^*$. As a rule, for problems where the dimension of the solution space Y is greater than that of the problem space X, backward stability is rare. □

Example 15.3. Suppose we use $\oplus$ to compute $x + 1$, given $x \in \mathbb{C}$: $\tilde{f}(x) = \text{fl}(x) \oplus 1$. This algorithm is stable but not backward stable. The reason is that for $x \approx 0$, the addition $\oplus$ will introduce absolute errors of size $O(\epsilon_{\text{machine}})$. Relative to the size of x, these are unbounded, so they cannot be interpreted as caused by small relative perturbations in the data. This example indicates that backward stability is a rather special property, a reasonable goal in some contexts but not others. Note that if the problem had been to compute $x + y$ for data x and y, then the algorithm would have been backward stable. □

Example 15.4. What is it reasonable to expect of a computer program or calculator that computes $\sin x$ or $\cos x$? Again the answer is stability, not backward stability. For $\cos x$, this follows from the fact that $\cos 0 \neq 0$, as in the previous example. For both $\sin x$ and $\cos x$, backward stability is also ruled out by the fact that the function has derivative equal to zero at certain points. For example, suppose we evaluate $f(x) = \sin x$ on a computer for $x = \pi/2 - \delta$, $0 < \delta \ll 1$. Suppose we are lucky enough to get as a computed result the exactly correct answer, rounded to the floating point system: $\tilde{f}(x) = \mathrm{fl}(\sin x)$. Since $f'(x) = \cos x \approx \delta$, we have $\tilde{f}(x) = f(\tilde{x})$ for some $\tilde{x}$ with $\tilde{x} - x \approx (\tilde{f}(x) - f(x))/\delta = O(\epsilon_{\text{machine}}/\delta)$. Since δ can be arbitrarily small, this backward error is not of magnitude $O(\epsilon_{\text{machine}})$. □

An Unstable Algorithm

These are toy examples. Here is a more substantial one: the use of the characteristic polynomial to find the eigenvalues of a matrix.

Since z is an eigenvalue of A if and only if $p(z) = 0$, where $p(z)$ is the characteristic polynomial $\det(zI - A)$, the roots of p are the eigenvalues of A (Lecture 24). This suggests a method for computing the eigenvalues:

1. Find the coefficients of the characteristic polynomial,
2. Find its roots.

This algorithm is not only backward unstable but unstable, and it should not be used. Even in cases where extracting eigenvalues is a well-conditioned problem, it may produce answers that have relative errors vastly larger than $\epsilon_{\text{machine}}$.

The instability is revealed in the rootfinding of the second step. As we saw in Example 12.5, the problem of finding the roots of a polynomial, given the coefficients, is generally ill-conditioned. It follows that small errors in the coefficients of the characteristic polynomial will tend to be amplified when finding roots, even if the rootfinding is done to perfect accuracy.

For example, suppose $A = I$, the 2×2 identity matrix. The eigenvalues of A are insensitive to perturbations of the entries, and a stable algorithm should be able to compute them with errors $O(\epsilon_{\text{machine}})$. However, the algorithm described above produces errors on the order of $\sqrt{\epsilon_{\text{machine}}}$. To explain this, we note that the characteristic polynomial is $x^2 - 2x + 1$, just as in Example 12.5. When the coefficients of this polynomial are computed, they can be expected to have errors on the order of $\epsilon_{\text{machine}}$, and these can cause the roots to change by order $\sqrt{\epsilon_{\text{machine}}}$. For example, if $\epsilon_{\text{machine}} = 10^{-16}$, the roots of the computed characteristic polynomial can be perturbed from the actual eigenvalues by approximately 10^{-8}, a loss of eight digits of accuracy.

Before you try this computation for yourself, we must be a little more honest. If you use the algorithm just described to compute the eigenvalues of

the 2×2 identity matrix, you will probably find that there are no errors at all, because the coefficients and roots of $x^2 - 2x + 1$ are small integers that will be represented exactly on your computer. However, if the experiment is done on a slightly perturbed matrix, such as

$$A = \begin{bmatrix} 1 + 10^{-14} & 0 \\ 0 & 1 \end{bmatrix},$$

the computed eigenvalues will differ from the actual ones by the expected order $\sqrt{\epsilon_{\text{machine}}}$. Try it!

Accuracy of a Backward Stable Algorithm

Suppose we have a backward stable algorithm $\tilde{f}$ for a problem $f : X \to Y$. Will the results it delivers be accurate? The answer depends on the condition number $\kappa = \kappa(x)$ of f. If $\kappa(x)$ is small, the results will be accurate in the relative sense, but if it is large, the accuracy will suffer proportionately.

Theorem 15.1. *Suppose a backward stable algorithm is applied to solve a problem $f : X \to Y$ with condition number κ on a computer satisfying the axioms* (13.5) *and* (13.7). *Then the relative errors satisfy*

$$\frac{\|\tilde{f}(x) - f(x)\|}{\|f(x)\|} = O(\kappa(x)\,\epsilon_{\text{machine}}). \tag{15.1}$$

Proof. By the definition (14.5) of backward stability, we have $\tilde{f}(x) = f(\tilde{x})$ for some $\tilde{x} \in X$ satisfying

$$\frac{\|\tilde{x} - x\|}{\|x\|} = O(\epsilon_{\text{machine}}).$$

By the definition (12.5) of $\kappa(x)$, this implies

$$\frac{\|\tilde{f}(x) - f(x)\|}{\|f(x)\|} \;\le\; (\kappa(x) + o(1))\,\frac{\|\tilde{x} - x\|}{\|x\|}, \tag{15.2}$$

where $o(1)$ denotes a quantity that converges to zero as $\epsilon_{\text{machine}} \to 0$. Combining these bounds gives (15.1). □

Backward Error Analysis

The process we have just carried out in proving Theorem 15.1 is known as *backward error analysis*. We obtained an accuracy estimate by two steps. One step was to investigate the condition of the problem. The other was to investigate the stability of the algorithm. Our conclusion was that if the algorithm is stable, then the final accuracy reflects that condition number.

Mathematically, this is straightforward, but it is certainly not the first idea an unprepared person would think of if called upon to analyze a numerical algorithm. The first idea would be *forward error analysis*. Here, the rounding errors introduced at each step of the calculation are estimated, and somehow, a total is maintained of how they may compound from step to step.

Experience has shown that for most of the algorithms of numerical linear algebra, forward error analysis is harder to carry out than backward error analysis. With the benefit of hindsight, it is not hard to explain why this is so. Suppose a tried-and-true algorithm is used, say, to solve $Ax = b$ on a computer. It is an established fact (see Lecture 22) that the results obtained will be consistently less accurate when A is ill-conditioned. Now, how could a forward error analysis capture this phenomenon? The condition number of A is so global a property as to be more or less invisible at the level of the individual floating point operations involved in solving $Ax = b$. (We dramatize this by an example in the next lecture.) Yet one way or another, the forward analysis will have to detect that condition number if it is to end up with a correct result.

In short, it is an established fact that the best algorithms for most problems do no better, in general, than to compute exact solutions for slightly perturbed data. Backward error analysis is a method of reasoning fitted neatly to this backward reality.

Exercises

15.1. Each of the following problems describes an algorithm implemented on a computer satisfying the axioms (13.5) and (13.7). For each one, state whether the algorithm is *backward stable*, *stable but not backward stable*, or *unstable*, and prove it or at least give a reasonably convincing argument. Be sure to follow the definitions as given in the text.

(a) Data: $x \in \mathbb{C}$. Solution: $2x$, computed as $x \oplus x$.

(b) Data: $x \in \mathbb{C}$. Solution: x^2, computed as $x \otimes x$.

(c) Data: $x \in \mathbb{C} \backslash \{0\}$. Solution: 1, computed as $x \oslash x$. (A machine satisfying (13.6) will give exactly the right answer, but our definitions are based on the weaker condition (13.7).)

(d) Data: $x \in \mathbb{C}$. Solution: 0, computed as $x \ominus x$. (Again, a real machine may do better than our definitions based on (13.7).)

(e) Data: none. Solution: e, computed by summing $\sum_{k=0}^{\infty} 1/k!$ from left to right using $\otimes$ and $\oplus$, stopping when a summand is reached of magnitude $< \epsilon_{\text{machine}}$.

(f) Data: none. Solution: e, computed by the same algorithm as above except with the series summed from right to left.

(g) Data: none. Solution: π, computed by doing an exhaustive search to find the smallest floating point number x in the interval $[3,4]$ such that $s(x) \otimes s(x') \leq 0$. Here $s(x)$ is an algorithm that calculates $\sin(x)$ stably in the given interval, and x' denotes the next floating point number after x in the floating point system.

15.2. Consider an algorithm for the problem of computing the (full) SVD of a matrix. The data for this problem is a matrix A, and the solution is three matrices U (unitary), Σ (diagonal), and V (unitary) such that $A = U\Sigma V^*$. (We are speaking here of explicit matrices U and V, not implicit representations as products of reflectors.)

(a) Explain what it would mean for this algorithm to be backward stable.

(b) In fact, for a simple reason, this algorithm cannot be backward stable. Explain.

(c) Fortunately, the standard algorithms for computing the SVD (Lecture 31) are stable. Explain what stability means for such an algorithm.

Lecture 16. Stability of Householder Triangularization

In this lecture we see backward error analysis in action. First we observe in a MATLAB experiment the remarkable phenomenon of backward stability of Householder triangularization. We then consider how the triangularization step can be combined with other backward stable pieces to obtain a stable algorithm for solving $Ax = b$.

Experiment

Householder factorization is a backward stable algorithm for computing QR factorizations. We can illustrate this by a MATLAB experiment carried out in IEEE double precision arithmetic, $\epsilon_{\text{machine}} \approx 1.11 \times 10^{-16}$.

`R = triu(randn(50));`	Set R to a 50×50 upper-triangular matrix with normal random entries.
`[Q,X] = qr(randn(50));`	Set Q to a random orthogonal matrix by orthogonalizing a random matrix.
`A = Q*R;`	Set A to the product QR, up to rounding errors.
`[Q2,R2] = qr(A);`	Compute QR factorization $A \approx Q_2 R_2$ by Householder triangularization.

The purpose of these four lines of MATLAB is to construct a matrix with a known QR factorization, $A = QR$, which can then be compared with the QR factorization $A = Q_2R_2$ computed by Householder triangularization. Actually, because of rounding errors, the QR factors of the computed matrix A are not exactly Q and R. However, for the purposes of this experiment they are close enough. The results about to be presented would not be significantly different if A were exactly equal to QR (which we could achieve, in effect, by calculating $A = QR$ in higher precision arithmetic on the computer).

For Q_2 and R_2, as it happens, are very far from exact:

```
norm(Q2-Q)                  How accurate is Q2?
    ans = 0.00889
norm(R2-R)/norm(R)          How accurate is R2?
    ans = 0.00071
```

These errors are huge! Our calculations have been done with sixteen digits of accuracy, yet the final results are accurate to only two or three digits. The individual rounding errors have been amplified by factors on the order of 10^{13}. (Note that the computed Q_2 is close enough to Q to indicate that changes in the signs of the columns cannot be responsible for any of the errors. If you try this experiment and get entirely different results, it may be that you need to multiply the columns of Q and rows of R by appropriate factors ± 1.)

We seem to have lost twelve digits of accuracy. But now, an astonishing thing happens when we multiply these inaccurate matrices Q_2 and R_2:

```
norm(A-Q2*R2)/norm(A)       How accurate is Q2R2?
    ans = 1.432e-15
```

The product Q_2R_2 is accurate to a full fifteen digits! The errors in Q_2 and R_2 must be "diabolically correlated," as Wilkinson used to say. To one part in 10^{12}, they cancel out in the product Q_2R_2.

To highlight how special this accuracy of Q_2R_2 is, let us construct another pair of matrices Q_3 and R_3 that are equally accurate approximations to Q and R, and multiply them.

```
Q3 = Q+1e-4*randn(50);      Set Q3 to a random perturbation of Q
                            that is closer to Q than Q2 is.
R3 = R+1e-4*randn(50);      Set R3 to a random perturbation of R
                            that is closer to R than R2 is.
norm(A-Q3*R3)/norm(A)       How accurate is Q3R3?
    ans = 0.00088
```

This time, the error in the product is huge. Q_2 is no better than Q_3, and R_2 is no better than R_3, but Q_2R_2 is twelve orders of magnitude better than Q_3R_3.

In this experiment, we did not take the trouble to make R_3 upper-triangular or Q_3 orthogonal, but there would have been little difference had we done so.

The errors in Q_2 and R_2 are *forward errors*. In general, a large forward error can be the result of an ill-conditioned problem or an unstable algorithm (Theorem 15.1). In our experiment, they are due to the former. As a rule, the sequences of column spaces of a random triangular matrix are exceedingly ill-conditioned as a function of the entries of the matrix.

The error in Q_2R_2 is the *backward error* or *residual*. The smallness of this error suggests that Householder triangularization is backward stable.

Theorem

In fact, Householder triangularization is backward stable for all matrices A and all computers satisfying (13.5) and (13.7). We shall now state a theorem to this effect. As with most stability results in this book, we shall not give a proof.

Our result will take the form

$$\tilde{Q}\tilde{R} = A + \delta A, \tag{16.1}$$

where δA is small. In words, the computed Q times the computed R equals a small perturbation of the given matrix A. However, a subtlety arises in the way we shall use these symbols. By $\tilde{R}$, we mean just what one would expect: the upper-triangular matrix that is constructed by Householder triangularization in floating point arithmetic. By $\tilde{Q}$, however, we mean a certain matrix that is *exactly unitary*. Recall that Q is equal to the product $Q = Q_1Q_2 \cdots Q_n$ (10.7), where Q_k is the Householder reflector defined by the vector v_k (10.4) determined at the kth step of Algorithm 10.1. In the floating point computation, we obtain instead a sequence of vectors $\tilde{v}_k$. Let $\tilde{Q}_k$ denote the *exactly unitary* reflector defined—mathematically, not on the computer—by the floating point vector $\tilde{v}_k$. Now define

$$\tilde{Q} = \tilde{Q}_1\tilde{Q}_2 \cdots \tilde{Q}_n. \tag{16.2}$$

This exactly unitary matrix $\tilde{Q}$ will take the role of our "computed Q." This definition may seem odd at first, but it is the natural one. In applications, as discussed in Lecture 10, the matrix Q is generally not formed explicitly anyway, so it would not be useful to define a "computed Q" of the more obvious kind. It is the vectors $\tilde{v}_k$ that are formed explicitly, and these are what enter into (16.2).

Here is the theorem that explains our MATLAB experiment.

Theorem 16.1. *Let the QR factorization $A = QR$ of a matrix $A \in \mathbb{C}^{m\times n}$ be computed by Householder triangularization (Algorithm* 10.1*) on a computer satisfying the axioms* (13.5) *and* (13.7)*, and let the computed factors $\tilde{Q}$ and $\tilde{R}$*

be defined as indicated above. Then we have

$$\tilde{Q}\tilde{R} = A + \delta A, \qquad \frac{\|\delta A\|}{\|A\|} = O(\epsilon_{\text{machine}}) \tag{16.3}$$

for some $\delta A \in \mathbb{C}^{m\times n}$.

As always in this book, the expression $O(\epsilon_{\text{machine}})$ in (16.3) has the precise meaning discussed in Lecture 14. The bound holds as $\epsilon_{\text{machine}} \to 0$, uniformly for all matrices A of any fixed dimensions m and n, but not uniformly with respect to m and n. Because all norms on a finite-dimensional vector space are equivalent, we need not specify the norm (Theorem 14.1).

Analyzing an Algorithm to Solve $Ax = b$

We have seen that Householder triangularization is backward stable but not always accurate in the forward sense. (The same is true of most of the matrix factorizations of numerical linear algebra.) Now, QR factorization is generally not an end in itself, but a means to other ends such as solution of a system of equations, a least squares problem, or an eigenvalue problem. Is its backward stability enough to make it a satisfactory piece of a larger algorithm? That is, is accuracy of the product QR enough for applications, or do we need accuracy of Q and R individually?

The happy answer is that accuracy of QR is indeed enough for most purposes. We can show this by surprisingly simple arguments.

The example we shall consider is the use of Householder triangularization to solve a nonsingular $m \times m$ linear system $Ax = b$. This idea was discussed at the end of Lecture 7. Here is a more complete statement of that algorithm.

Algorithm 16.1. Solving $Ax = b$ by QR Factorization

$QR = A$	Factor A into QR by Algorithm 10.1, with Q represented as the product of reflectors.
$y = Q^*b$	Construct Q^*b by Algorithm 10.2.
$x = R^{-1}y$	Solve the triangular system $Rx = y$ by back substitution (Algorithm 17.1).

This algorithm is backward stable, and proving this is straightforward, given that each of the three steps is itself backward stable. Here, we shall state backward stability results for the three steps, without proof, and then give the details of how they can be combined.

The first step of Algorithm 16.1 is QR factorization of A, leading to computed matrices $\tilde{R}$ and $\tilde{Q}$. The backward stability of this process has already been expressed by (16.3).

The second step is computation of $\tilde{Q}^*b$ by Algorithm 10.2. (Note that for the purposes of this argument we do not write Q^*b, for at this point of the

computation, the first step is complete, and the matrix it has produced is not Q but $\tilde{Q}$.) When $\tilde{Q}^*b$ is computed by Algorithm 10.2, rounding errors will be made, so the result will not be exactly $\tilde{Q}^*b$. Instead it will be some vector $\tilde{y}$. It can be shown that this vector satisfies the following backward stability estimate:

$$(\tilde{Q} + \delta Q)\tilde{y} = b, \qquad \|\delta Q\| = O(\epsilon_{\text{machine}}). \tag{16.4}$$

Like (16.3), this equality is exact. In words, the result of applying the Householder reflectors in floating point arithmetic is exactly equivalent to multiplying b by a slightly perturbed matrix, $(\tilde{Q} + \delta Q)^{-1}$.

The final step of Algorithm 16.1 is back substitution to compute $\tilde{R}^{-1}\tilde{y}$. (Again, it is $\tilde{R}$ and $\tilde{y}$ that are available at this stage; R and y have nothing to do with it.) In this step new rounding errors will be introduced, but once more, the computation is backward stable. This time the estimate takes the form

$$(\tilde{R} + \delta R)\tilde{x} = \tilde{y}, \qquad \frac{\|\delta R\|}{\|\tilde{R}\|} = O(\epsilon_{\text{machine}}). \tag{16.5}$$

As always, the equality on the left is exact. It asserts that the floating point result $\tilde{x}$ is the exact solution of a slight perturbation of the system $\tilde{R}x = \tilde{y}$.

In the next lecture we shall derive (16.5) in full detail—more interesting than it sounds! The result is stated as Theorem 17.1.

Now, taking (16.3)–(16.5) as given, here is the theorem that results. This is typical of backward stability theorems that can be proved for many of the algorithms of numerical linear algebra.

Theorem 16.2. *Algorithm* 16.1 *is backward stable, satisfying*

$$(A + \Delta A)\tilde{x} = b, \qquad \frac{\|\Delta A\|}{\|A\|} = O(\epsilon_{\text{machine}}) \tag{16.6}$$

for some $\Delta A \in \mathbb{C}^{m\times m}$.

Proof. Composing (16.4) and (16.5), we have

$$b \;=\; (\tilde{Q} + \delta Q)(\tilde{R} + \delta R)\tilde{x} \;=\; [\tilde{Q}\tilde{R} + (\delta Q)\tilde{R} + \tilde{Q}(\delta R) + (\delta Q)(\delta R)]\tilde{x}.$$

Therefore, by (16.3),

$$b \;=\; [A + \delta A + (\delta Q)\tilde{R} + \tilde{Q}(\delta R) + (\delta Q)(\delta R)]\tilde{x}.$$

This equation has the form

$$b = (A + \Delta A)\tilde{x},$$

where ΔA is a sum of four terms. To establish (16.6), we must show that each of these terms is small relative to A.

Since $\tilde{Q}\tilde{R} = A + \delta A$ and $\tilde{Q}$ is unitary, we have

$$\frac{\|\tilde{R}\|}{\|A\|} \;\le\; \|\tilde{Q}^*\| \, \frac{\|A + \delta A\|}{\|A\|} \;=\; O(1)$$

as $\epsilon_{\text{machine}} \to 0$, by (16.3). (It is $1 + O(\epsilon_{\text{machine}})$ if $\|\cdot\| = \|\cdot\|_2$, but we have made no assumptions about $\|\cdot\|$.) This gives us

$$\frac{\|(\delta Q)\tilde{R}\|}{\|A\|} \;\le\; \|\delta Q\| \, \frac{\|\tilde{R}\|}{\|A\|} \;=\; O(\epsilon_{\text{machine}})$$

by (16.4). Similarly,

$$\frac{\|\tilde{Q}(\delta R)\|}{\|A\|} \;\le\; \|\tilde{Q}\| \, \frac{\|\delta R\|}{\|\tilde{R}\|} \, \frac{\|\tilde{R}\|}{\|A\|} \;=\; O(\epsilon_{\text{machine}})$$

by (16.5). Finally,

$$\frac{\|(\delta Q)(\delta R)\|}{\|A\|} \;\le\; \|\delta Q\| \, \frac{\|\delta R\|}{\|A\|} \;=\; O(\epsilon_{\text{machine}}^2).$$

The total perturbation ΔA thus satisfies

$$\frac{\|\Delta A\|}{\|A\|} \;\le\; \frac{\|\delta A\|}{\|A\|} + \frac{\|(\delta Q)\tilde{R}\|}{\|A\|} + \frac{\|\tilde{Q}(\delta R)\|}{\|A\|} + \frac{\|(\delta Q)(\delta R)\|}{\|A\|} \;=\; O(\epsilon_{\text{machine}}),$$

as claimed. □

Combining Theorems 12.2, 15.1, and 16.2 gives the following result about accuracy of solutions of $Ax = b$.

Theorem 16.3. *The solution $\tilde{x}$ computed by Algorithm* 16.1 *satisfies*

$$\frac{\|\tilde{x} - x\|}{\|x\|} = O(\kappa(A)\,\epsilon_{\text{machine}}). \tag{16.7}$$

Exercises

16.1. (a) Let unitary matrices $Q_1, \ldots, Q_k \in \mathbb{C}^{m,m}$ be fixed and consider the problem of computing, for $A \in \mathbb{C}^{m \times n}$, the product $B = Q_k \cdots Q_1 A$. Let the computation be carried out from right to left by straightforward floating point operations on a computer satisfying (13.5) and (13.7). Show that this algorithm is backward stable. (Here A is thought of as data that can be perturbed; the matrices Q_j are fixed and not to be perturbed.)

(b) Give an example to show that this result no longer holds if the unitary matrices Q_j are replaced by arbitrary matrices $X_j \in \mathbb{C}^{m\times m}$.

16.2. The idea of this exercise is to carry out an experiment analogous to the one described in this lecture, but for the SVD instead of QR factorization.

(a) Write a MATLAB program that constructs a 50×50 matrix `A=U*S*V'`, where `U` and `V` are random orthogonal matrices and `S` is a diagonal matrix whose diagonal entries are random uniformly distributed numbers in $[0, 1]$, sorted into nonincreasing order. Have your program compute `[U2,S2,V2] = svd(A)` and the norms of `U-U2`, `V-V2`, `S-S2`, and `A-U2*S2*V2'`. Do this for five matrices `A` and comment on the results. (Hint: Plots of `diag(U2'*U)` and `diag(V2'*V)` may be informative.)

(b) Fix the signs in your computed SVD so that the difficulties of (a) go away. Run the program again for five random matrices and comment on the various norms. Do they have a connection with `cond(A)`?

(c) Replace the diagonal entries of `S` by their sixth powers and repeat (b). Do you see significant differences between the results of this exercise and those of the experiment for QR factorization?

Lecture 17. Stability of Back Substitution

One of the easiest problems of numerical linear algebra is the solution of a triangular system of equations. The standard algorithm is successive substitution, called back substitution when the system is upper-triangular. Here we show in full detail that this algorithm is backward stable, obtaining quantitative bounds on the effects of rounding errors, with no "$O(\epsilon_{\text{machine}})$".

Triangular Systems

We have seen that a general system of equations $Ax = b$ can be reduced to an upper-triangular system $Rx = y$ by QR factorization. Lower- and upper-triangular systems also arise in Gaussian elimination, in Cholesky factorization, and in numerous other computations of numerical linear algebra. These systems are easily solved by a process of successive substitution, called *forward substitution* if the system is lower-triangular and *back substitution* if it is upper-triangular. Although the two cases are mathematically identical, for definiteness, we treat back substitution in this lecture.

Suppose we wish to solve $Rx = b$, that is,

$$\begin{bmatrix} r_{11} & r_{12} & \cdots & r_{1m} \\ & r_{22} & & \\ & & \ddots & \vdots \\ & & & r_{mm} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix}, \tag{17.1}$$

where $b \in \mathbb{C}^m$ and $R \in \mathbb{C}^{m\times m}$, nonsingular and upper-triangular, are given, and $x \in \mathbb{C}^m$ is unknown. We can do this by solving for the components of x one after another, beginning with x_m and finishing with x_1. For later convenience we write the algorithm as a sequence of formulas rather than a loop.

Algorithm 17.1. Back Substitution

$$
\begin{aligned}
x_m &= b_m / r_{mm} \\
x_{m-1} &= (b_{m-1} - x_m r_{m-1,m}) / r_{m-1,m-1} \\
x_{m-2} &= (b_{m-2} - x_{m-1} r_{m-2,m-1} - x_m r_{m-2,m}) / r_{m-2,m-2} \\
&\vdots \\
x_j &= \left(b_j - \sum_{k=j+1}^{m} x_k r_{jk} \right) \Big/ r_{jj}
\end{aligned}
$$

The structure is triangular, with a subtraction and a multiplication at each position. The operation count is accordingly twice the area of an $m \times m$ triangle:

$$\text{Work for back substitution:} \quad \sim m^2 \text{ flops.} \tag{17.2}$$

Backward Stability Theorem

In the last lecture, back substitution appeared as one of three steps in the solution of $Ax = b$ by QR factorization. In (16.3)–(16.5) we asserted that each of these steps is backward stable, but we did not prove these claims. In this lecture we shall fill one of these gaps by deriving a bound that implies (16.5). Our argument is an example of how proofs of backward stability are organized. This will be the only case in this book in which we give all the details of such a proof.

Before we can prove that Algorithm 17.1 is backward stable, however, we must pin down one detail of the algorithm that is not specified by the formulas as written. Let us decide, arbitrarily, that in the expressions in parentheses above, the subtractions will be carried out from left to right. (Other orders are also stable; only the details of the estimates are different.) Now we can state our theorem.

Theorem 17.1. *Let Algorithm* 17.1 *be applied to a problem* (17.1) *consisting of floating point numbers on a computer satisfying* (13.7). *This algorithm is backward stable in the sense that the computed solution* $\tilde{x} \in \mathbb{C}^m$ *satisfies*

$$(R + \delta R)\tilde{x} = b \tag{17.3}$$

for some upper-triangular $\delta R \in \mathbb{C}^{m\times m}$ *with*

$$\frac{\|\delta R\|}{\|R\|} = O(\epsilon_{\text{machine}}). \tag{17.4}$$

Specifically, for each i, j,

$$\frac{|\delta r_{ij}|}{|r_{ij}|} \le m\epsilon_{\text{machine}} + O(\epsilon_{\text{machine}}^2). \tag{17.5}$$

In (17.5) and throughout this lecture, we continue to use the convention of (14.12) that if the denominator is zero, the numerator is implicitly asserted to be zero also (for all sufficiently small $\epsilon_{\text{machine}}$).

To keep the ideas clear and interesting, our proof will be most leisurely.

$m = 1$

According to (17.3), our task is to express every floating point error as a perturbation of the input. Let us begin with the simplest case, where R is of dimension 1×1. Back substitution in this case consists of a single step,

$$\tilde{x}_1 = b_1 \oslash r_{11}.$$

(Recall from Lecture 13 that $\oslash$, $\otimes$, $\oplus$, and $\ominus$ denote floating point operations.) The axiom (13.7) for $\oslash$ guarantees that the computed solution is close to correct:

$$\tilde{x}_1 = \frac{b_1}{r_{11}}(1+\epsilon_1), \qquad |\epsilon_1| \le \epsilon_{\text{machine}}.$$

However, we would like to express the error as if it resulted from a perturbation in R. To this end, we set $\epsilon_1' = -\epsilon_1/(1+\epsilon_1)$, whereupon the formula becomes

$$\tilde{x}_1 = \frac{b_1}{r_{11}(1+\epsilon_1')}, \qquad |\epsilon_1'| \le \epsilon_{\text{machine}} + O(\epsilon_{\text{machine}}^2). \tag{17.6}$$

Note that ϵ_1' is equal to $-\epsilon_1$ plus a term of order ${\epsilon_1}^2$. We can freely move small relative perturbations from numerators to denominators or vice versa, and the result changes by terms of order $\epsilon_{\text{machine}}^2$ (Exercise 14.2(b)).

In (17.6), the equality is exact; the division is mathematical, not floating point. The formula states that 1×1 back substitution is backward stable, for $\tilde{x}_1$ is exactly the correct solution to a perturbed problem, namely

$$(r_{11} + \delta r_{11})\tilde{x}_1 = b_1,$$

with $\delta r_{11} = \epsilon_1' r_{11}$; hence

$$\frac{|\delta r_{11}|}{|r_{11}|} \le \epsilon_{\text{machine}} + O(\epsilon_{\text{machine}}^2).$$

$m = 2$

The 2×2 case is slightly less trivial. Suppose we have an upper-triangular matrix $R \in \mathbb{C}^{2\times 2}$ and a vector $b \in \mathbb{C}^2$. The computation of $\tilde{x} \in \mathbb{C}^2$ proceeds in two steps. The first is the same as in the 1×1 case:

$$\tilde{x}_2 = b_2 \oslash r_{22} = \frac{b_2}{r_{22}(1+\epsilon_1)}, \quad |\epsilon_1| \le \epsilon_{\text{machine}} + O(\epsilon^2_{\text{machine}}). \tag{17.7}$$

The second step is defined by the formula

$$\tilde{x}_1 \;=\; (b_1 \ominus (\tilde{x}_2 \otimes r_{12})) \oslash r_{11}.$$

To establish backward stability, we must express the errors in these three floating point operations as perturbations in the entries r_{ij}.

The multiplication is easy; we use the axiom (13.7) directly, interpreting the floating point multiplication as a perturbation in r_{12}:

$$\tilde{x}_1 \;=\; (b_1 \ominus \tilde{x}_2 r_{12}(1+\epsilon_2)) \oslash r_{11}, \quad |\epsilon_2| \le \epsilon_{\text{machine}}.$$

The division and subtraction are more subtle. First, we write the formula with exact mathematics according to (13.7):

$$\tilde{x}_1 \;=\; (b_1 - \tilde{x}_2 r_{12}(1+\epsilon_2))(1+\epsilon_3) \oslash r_{11} \tag{17.8}$$

$$\;=\; \frac{(b_1 - \tilde{x}_2 r_{12}(1+\epsilon_2))(1+\epsilon_3)}{r_{11}}(1+\epsilon_4). \tag{17.9}$$

Here (13.7) guarantees $|\epsilon_3|, |\epsilon_4| \le \epsilon_{\text{machine}}$. Now we shift the ϵ_3 and ϵ_4 terms from the numerator to the denominator, as before. This gives

$$\tilde{x}_1 \;=\; \frac{b_1 - \tilde{x}_2 r_{12}(1+\epsilon_2)}{r_{11}(1+\epsilon_3')(1+\epsilon_4')},$$

with $|\epsilon_3'|, |\epsilon_4'| \le \epsilon_{\text{machine}} + O(\epsilon^2_{\text{machine}})$, or equivalently,

$$\tilde{x}_1 \;=\; \frac{b_1 - \tilde{x}_2 r_{12}(1+\epsilon_2)}{r_{11}(1+2\epsilon_5)}, \tag{17.10}$$

with $|\epsilon_5| \le \epsilon_{\text{machine}} + O(\epsilon^2_{\text{machine}})$. This formula states that $\tilde{x}_1$ would be exactly correct if r_{22}, r_{12}, and r_{11} were perturbed by factors $(1+\epsilon_1)$, $(1+\epsilon_2)$, and $(1+2\epsilon_5)$, respectively. These perturbations can be summarized by the equation

$$(R + \delta R)\tilde{x} = b,$$

where the entries δr_{ij} of δR satisfy

$$\begin{bmatrix} |\delta r_{11}|/|r_{11}| & |\delta r_{12}|/|r_{12}| \\ & |\delta r_{22}|/|r_{22}| \end{bmatrix} = \begin{bmatrix} 2|\epsilon_5| & |\epsilon_2| \\ & |\epsilon_1| \end{bmatrix} \le \begin{bmatrix} 2 & 1 \\ & 1 \end{bmatrix} \epsilon_{\text{machine}} + O(\epsilon^2_{\text{machine}}).$$

(The "$\le$" here and in similar results below is to be interpreted entrywise.) This formula guarantees $\|\delta R\|/\|R\| = O(\epsilon_{\text{machine}})$ in any matrix norm and thus that 2×2 back substitution is backward stable.

$m = 3$

The analysis for a 3×3 matrix includes all the reasoning necessary for the general case. The first two steps are the same as before:

$$\tilde{x}_3 = b_3 \oslash r_{33} = \frac{b_3}{r_{33}(1+\epsilon_1)}, \tag{17.11}$$

$$\tilde{x}_2 = (b_2 \ominus (\tilde{x}_3 \otimes r_{23})) \oslash r_{22} = \frac{b_2 - \tilde{x}_3 r_{23}(1+\epsilon_2)}{r_{22}(1+2\epsilon_3)}, \tag{17.12}$$

where

$$\begin{bmatrix} 2|\epsilon_3| & |\epsilon_2| \\ & |\epsilon_1| \end{bmatrix} \leq \begin{bmatrix} 2 & 1 \\ & 1 \end{bmatrix} \epsilon_{\text{machine}} + O(\epsilon_{\text{machine}}^2).$$

The third step involves the computation

$$\tilde{x}_1 = [(b_1 \ominus (\tilde{x}_2 \otimes r_{12})) \ominus (\tilde{x}_3 \otimes r_{13})] \oslash r_{11}. \tag{17.13}$$

We convert the two $\otimes$ operations in (17.13) to mathematical multiplication by introducing perturbations ϵ_4 and ϵ_5:

$$\tilde{x}_1 = [(b_1 \ominus \tilde{x}_2 r_{12}(1+\epsilon_4)) \ominus \tilde{x}_3 r_{13}(1+\epsilon_5)] \oslash r_{11}.$$

We convert the $\ominus$ operations to mathematical subtractions via perturbations ϵ_6 and ϵ_7:

$$\tilde{x}_1 = [(b_1 - \tilde{x}_2 r_{12}(1+\epsilon_4))(1+\epsilon_6) - \tilde{x}_3 r_{13}(1+\epsilon_5)](1+\epsilon_7) \oslash r_{11}.$$

Finally, the $\oslash$ is eliminated using ϵ_8; let us immediately replace this by ϵ_8' with $|\epsilon_8| \leq \epsilon_{\text{machine}} + O(\epsilon_{\text{machine}}^2)$ and put the result in the denominator:

$$\tilde{x}_1 = \frac{[(b_1 - \tilde{x}_2 r_{12}(1+\epsilon_4))(1+\epsilon_6) - \tilde{x}_3 r_{13}(1+\epsilon_5)](1+\epsilon_7)}{r_{11}(1+\epsilon_8')}.$$

Now, the expression above has everything as we need it except the terms involving ϵ_6 and ϵ_7, which originated from operations $\ominus$. If these are distributed, they will affect the number b_1, whereas our aim is to perturb only the entries r_{ij}. The term involving ϵ_7 is easily dispatched: we change ϵ_7 to ϵ_7' and move it to the denominator as usual. The term involving ϵ_6 requires a new trick. We move it to the denominator too, but to keep the equality valid, we compensate by putting the new factor $(1+\epsilon_6')$ into the r_{13} term as well. Thus

$$\tilde{x}_1 = \frac{b_1 - \tilde{x}_2 r_{12}(1+\epsilon_4) - \tilde{x}_3 r_{13}(1+\epsilon_5)(1+\epsilon_6')}{r_{11}(1+\epsilon_6')(1+\epsilon_7')(1+\epsilon_8')}.$$

Now r_{13} has two perturbations of size at most $\epsilon_{\text{machine}}$, and r_{11} has three. In this formula, all the errors in the computation have been expressed as perturbations in the entries of R.

The result can be summarized as

$$(R + \delta R)\tilde{x} = b,$$

where the entries δr_{ij} satisfy

$$\begin{bmatrix} |\delta r_{11}|/|r_{11}| & |\delta r_{12}|/|r_{12}| & |\delta r_{13}|/|r_{13}| \\ & |\delta r_{22}|/|r_{22}| & |\delta r_{23}|/|r_{23}| \\ & & |\delta r_{33}|/|r_{33}| \end{bmatrix} \leq \begin{bmatrix} 3 & 1 & 2 \\ & 2 & 1 \\ & & 1 \end{bmatrix} \epsilon_{\text{machine}} + O(\epsilon_{\text{machine}}^2).$$

General m

The analysis in higher-dimensional cases is similar. For example, in the 5×5 case we obtain the componentwise bound

$$\frac{|\delta R|}{|R|} \leq \begin{bmatrix} 5 & 1 & 2 & 3 & 4 \\ & 4 & 1 & 2 & 3 \\ & & 3 & 1 & 2 \\ & & & 2 & 1 \\ & & & & 1 \end{bmatrix} \epsilon_{\text{machine}} + O(\epsilon_{\text{machine}}^2). \tag{17.14}$$

The entries of the matrix in this formula are obtained from three components. The multiplications $\tilde{x}_k r_{jk}$ introduce $\epsilon_{\text{machine}}$ perturbations in the pattern

$$\otimes: \quad \begin{bmatrix} 0 & 1 & 1 & 1 & 1 \\ & 0 & 1 & 1 & 1 \\ & & 0 & 1 & 1 \\ & & & 0 & 1 \\ & & & & 0 \end{bmatrix}. \tag{17.15}$$

The divisions by r_{kk} introduce perturbations in the pattern

$$\oslash: \quad \begin{bmatrix} 1 & & & & \\ & 1 & & & \\ & & 1 & & \\ & & & 1 & \\ & & & & 1 \end{bmatrix}. \tag{17.16}$$

Finally, the subtractions also occur in the pattern (17.15), and, due to the decision to compute from left to right, each one introduces a perturbation on the diagonal and at each position to the right. This adds up to the pattern

$$\ominus: \quad \begin{bmatrix} 4 & 0 & 1 & 2 & 3 \\ & 3 & 0 & 1 & 2 \\ & & 2 & 0 & 1 \\ & & & 1 & 0 \\ & & & & 0 \end{bmatrix}. \tag{17.17}$$

Adding (17.15), (17.16), and (17.17) produces the result in (17.14). This completes the proof of Theorem 17.1.

Remarks

The analysis leading to (17.14) is typical of backward error analysis for all kinds of floating point computations. The only raw ingredient is the floating point axiom (13.7), ensuring that each operation $\oplus$, $\ominus$, $\otimes$, or $\oslash$ (sometimes also floating point $\sqrt{\ }$) introduces a small relative error. One utilizes this axiom repeatedly and carefully, interpreting each error in the course of the calculation as an error in the initial data. Perturbations of order $\epsilon_{\text{machine}}$ are composed additively and moved freely between numerators and denominators, since the difference is of order $\epsilon_{\text{machine}}^2$.

More than one error bound can be derived for a given algorithm. In the present case, we could have perturbed b_j as well as r_{ij}, avoiding the need for the trickery represented in (17.17). On the other hand, a final result in which only R is perturbed is appealingly clean.

Equation (17.5) is a *componentwise* backward error bound, meaning that each entry r_{ij} is perturbed by a quantity that is small relative to itself, not just relative to the norm of R. For example, if $r_{ij} = 0$, this entry undergoes no perturbation at all: δR has the same sparsity pattern as R. Some algorithms of numerical linear algebra satisfy componentwise backward error estimates, and some do not; in the latter case we must settle for a weaker *normwise* estimate such as (17.4). In the early decades of numerical analysis after the Second World War, most error estimates were obtained in the normwise form, whereas in more recent years there has been a shift toward componentwise analysis, since such results are sharper and algorithms that satisfy componentwise bounds are less sensitive to scaling of variables. However, we shall not pursue these matters in this book.

We close with a comment about the relationship between quantitative bounds like (17.5) or (17.14) and those such as (17.4) that we express in the $O(\epsilon_{\text{machine}})$ notation (Exercise 17.1). Why do we not dispense with crude statements like (17.4)? The reason is that quantitative bounds must include factors such as $\sqrt{m}$ or m, and these are norm-dependent, unmemorable, and often pessimistic in practice anyway because of statistical cancellation. We prefer to avoid such complications by expressing most of our results in terms of $O(\epsilon_{\text{machine}})$—which is, most assuredly, the form in which numerical analysts remember them.

Exercises

17.1. For any particular choice of norm $\|\cdot\|$, the bound (17.5) implies a more quantitative normwise bound than (17.4). Derive such bounds for the norms (a) $\|\cdot\|_1$, (b) $\|\cdot\|_2$, (c) $\|\cdot\|_\infty$.

17.2. A triangular system (17.1) is solved by back substitution. Exactly what does Theorem 17.1 imply about the error $\|\tilde{x} - x\|$?

17.3. Let $L \in \mathbb{C}^{m\times m}$ be a unit lower-triangular matrix (i.e., with diagonal entries equal to 1). For convenience, write L in the form

$$L = \begin{bmatrix} 1 & & & & \\ -\ell_{2,1} & 1 & & & \\ -\ell_{3,1} & -\ell_{3,2} & 1 & & \\ \vdots & \vdots & & \ddots & \\ -\ell_{m,1} & -\ell_{m,2} & -\ell_{m,3} & \cdots & 1 \end{bmatrix},$$

and define $M = L^{-1}$.

(a) Derive a formula for m_{ij} (which may involve other entries of M). Which entries of L does m_{ij} depend on?

(b) Suppose the subdiagonal entries of L are independent random numbers ± 1 with equal probability. Fix k and define $\mu_1 = m_{kk}$, $\mu_2 = m_{k+1,k}$, $\mu_3 = m_{k+2,k}, \ldots$. Write down a system of recurrence relations with random coefficients for the numbers μ_j.

(c) Experiments show that random triangular matrices with entries ± 1 are exponentially ill-conditioned in the sense that if κ_m denotes the 2-norm condition number of a matrix of this kind of dimension m, then $\lim_{m\to\infty}(\kappa_m)^{1/m} = C$ for some constant $1 < C < 1.5$. (The limit process can be made precise in various ways, but we shall not go into the technicalities; think of it as holding "with probability 1.") Perform numerical experiments involving random matrices of various dimensions to estimate C to 10% accuracy or better.

(d) Larger scale experiments become feasible if the random matrices of (c) are replaced by the random sequences $\mu_1, \mu_2, \mu_3, \ldots$ of (b). Explain (without proof) why the constant C can also be obtained by considering these sequences, and carry out numerical experiments to estimate C to 1% accuracy or better.

Lecture 18. Conditioning of Least Squares Problems

The conditioning of least squares problems is a subtle topic, combining the conditioning of square systems of equations with the geometry of orthogonal projection. It is important because it has nontrivial implications for the stability of least squares algorithms.

Four Conditioning Problems

In this lecture we return to the linear least squares problem (11.2), illustrated again in Figure 18.1. We assume the matrix defining the problem is of full rank, and write throughout this lecture $\|\cdot\| = \|\cdot\|_2$:

$$\begin{aligned} &\text{Given } A \in \mathbb{C}^{m\times n} \text{ of full rank, } m \geq n,\ b \in \mathbb{C}^m, \\ &\text{find } x \in \mathbb{C}^n \text{ such that } \|b - Ax\| \text{ is minimized.} \end{aligned} \tag{18.1}$$

The solution x and the corresponding point $y = Ax$ that is closest to b in range(A) are given by

$$x = A^+ b, \qquad y = Pb, \tag{18.2}$$

where $A^+ \in \mathbb{C}^{n\times m}$ is the pseudoinverse (11.11) of A and $P = AA^+ \in \mathbb{C}^{m\times m}$ is the orthogonal projector onto range(A). We consider the conditioning of (18.1) with respect to perturbations. Just as the last lecture represented the most detailed analysis of stability in this book, the present lecture represents our most detailed analysis of conditioning. We pick the least squares problem

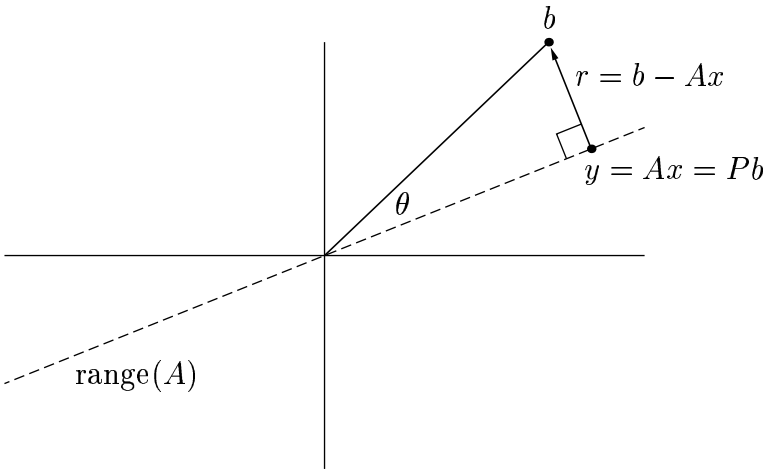


Figure 18.1. *The least squares problem (repetition of Figure* 11.3*).*

because the details are interesting and because they have an important practical consequence, to be discussed in the next lecture: the instability of the normal equations as a general purpose least squares algorithm.

Conditioning pertains to the sensitivity of solutions to perturbations in data. For (18.1), we shall investigate two choices of each. The data for the problem are the $m \times n$ matrix A and the m-vector b. The solution is either the coefficient vector x or the corresponding point $y = Ax$. Thus

$$\text{Data: } A, b, \qquad \text{Solution: } x, y.$$

Together, these two pairs of choices define four conditioning questions that we shall consider, all of which have application in certain contexts.

Theorem

The centerpiece of this lecture is Theorem 18.1, which provides answers to these questions. The results are expressed in terms of three dimensionless parameters that appear repeatedly in the analysis of least squares problems. The first is the condition number of A. For a square matrix, this is $\kappa(A) = \|A\|\,\|A^{-1}\|$, and in the rectangular case, the definition generalizes to (12.17),

$$\kappa(A) = \|A\|\,\|A^+\| = \frac{\sigma_1}{\sigma_n}. \tag{18.3}$$

The second is the angle θ marked in Figure 18.1, a measure of the closeness of the fit:

$$\theta = \cos^{-1}\frac{\|y\|}{\|b\|}. \tag{18.4}$$

The third is a measure of how much $\|y\|$ falls short of its maximum possible value, given $\|A\|$ and $\|x\|$:

$$\eta = \frac{\|A\|\,\|x\|}{\|y\|} = \frac{\|A\|\,\|x\|}{\|Ax\|}. \tag{18.5}$$

These parameters lie in the ranges

$$1 \le \kappa(A) < \infty, \qquad 0 \le \theta \le \pi/2, \qquad 1 \le \eta \le \kappa(A). \tag{18.6}$$

Theorem 18.1. *Let $b \in \mathbb{C}^m$ and $A \in \mathbb{C}^{m\times n}$ of full rank be fixed. The least squares problem* (18.1) *has the following* 2*-norm relative condition numbers* (12.5) *describing the sensitivities of y and x to perturbations in b and A:*

	y	x
b	$\dfrac{1}{\cos\theta}$	$\dfrac{\kappa(A)}{\eta\cos\theta}$
A	$\dfrac{\kappa(A)}{\cos\theta}$	$\kappa(A) + \dfrac{\kappa(A)^2\tan\theta}{\eta}$

The results in the first row are exact, being attained for certain perturbations δb, and the results in the second row are upper bounds.

In the special case $m = n$, (18.1) reduces to a square, nonsingular system of equations, with $\theta = 0$. In this case, the numbers in the second column of the theorem reduce to $\kappa(A)/\eta$ and $\kappa(A)$, which are the results (12.14) and (12.18) derived earlier, and the number in the lower-left position can be replaced by 0 (Exercise 18.4).

Transformation to a Diagonal Matrix

As a first step in the proof of Theorem 18.1, we note that the least squares problem becomes easier to analyze if we transform to a convenient choice of bases. Let A have an SVD of the form $A = U\Sigma V^*$, where Σ is an $m\times n$ diagonal matrix with positive diagonal entries. Since perturbations are measured in the 2-norm, their sizes are unaffected by a unitary change of basis, so the perturbation behavior of A is the same as that of Σ. Therefore, without loss of generality, we can deal with Σ directly. For the remainder of the discussion, we assume $A = \Sigma$ and write

$$A = \begin{bmatrix} \sigma_1 & & & \\ & \sigma_2 & & \\ & & \ddots & \\ & & & \sigma_n \\ & & & \end{bmatrix} = \begin{bmatrix} A_1 \\ 0 \end{bmatrix}. \tag{18.7}$$

Here A_1 is $n \times n$ and diagonal; the rest of A is zero.

The orthogonal projection of b onto range(A) is now a triviality. Write

$$b = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix},$$

where b_1 contains the first n entries of b. Then the projection $y = Pb$ is

$$y = \begin{bmatrix} b_1 \\ 0 \end{bmatrix}.$$

To find the corresponding x we can write $Ax = y$ as

$$\begin{bmatrix} A_1 \\ 0 \end{bmatrix} x = \begin{bmatrix} b_1 \\ 0 \end{bmatrix},$$

which implies

$$x = A_1^{-1} b_1. \tag{18.8}$$

From these formulas it is evident that the orthogonal projector and pseudoinverse are the block 2×2 and 1×2 matrices (see Exercise 11.1)

$$P = \begin{bmatrix} I & 0 \\ 0 & 0 \end{bmatrix}, \qquad A^+ = \begin{bmatrix} A_1^{-1} & 0 \end{bmatrix}. \tag{18.9}$$

Sensitivity of y to Perturbations in b

We begin with the simplest of our four conditioning results. By (18.2), the relationship between b and y is just the linear equation $y = Pb$. The Jacobian of this mapping is P itself, with $\|P\| = 1$ by (18.9). By (12.6) and (18.4), the condition number of y with respect to perturbations in b is accordingly

$$\kappa_{b \mapsto y} = \frac{\|P\|}{\|y\|/\|b\|} = \frac{1}{\cos\theta}.$$

This establishes the upper-left result of Theorem 18.1. The condition number is realized (that is, the supremum in (12.5) is attained) for perturbations δb with $\|P(\delta b)\| = \|\delta b\|$, which occurs when δb is zero except in the first n entries.

Sensitivity of x to Perturbations in b

The relationship between b and x is also linear, $x = A^+ b$, with Jacobian A^+. By (12.6), (18.4), and (18.5), the condition number of x with respect to perturbations in b is consequently

$$\kappa_{b \mapsto x} = \frac{\|A^+\|}{\|x\|/\|b\|} = \|A^+\| \frac{\|b\|}{\|y\|} \frac{\|y\|}{\|x\|} = \|A^+\| \frac{1}{\cos\theta} \frac{\|A\|}{\eta} = \frac{\kappa(A)}{\eta \cos\theta}.$$

This establishes the upper-right result of Theorem 18.1. Here, the condition number is realized by perturbations δb satisfying $\|A^+(\delta b)\| = \|A^+\| \, \|\delta b\| = \|\delta b\|/\sigma_n$, which occurs when δb is zero except in the nth entry (or perhaps also in other entries, if A has more than one singular value equal to σ_n).

Tilting the Range of A

The analysis of perturbations in A is a nonlinear problem and more subtle. We could proceed by calculating Jacobians algebraically, but instead, we shall take a geometric view. Our starting point is the observation that perturbations in A affect the least squares problem in two ways: they distort the mapping of $\mathbb{C}^n$ onto range(A), and they alter range(A) itself. Let us consider this latter effect for a moment.

We can visualize slight changes in range(A) as small "tiltings" of this space. The question is, what is the maximum angle of tilt $\delta\alpha$ that can be imparted by a small perturbation δA? The answer can be determined as follows. The image under A of the unit n-sphere is a hyperellipse that lies flat in range(A). To change range(A) as efficiently as possible, we grasp a point $p = Av$ on the hyperellipse (hence $\|v\| = 1$) and nudge it in a direction δp orthogonal to range(A). A matrix perturbation that achieves this most efficiently is $\delta A = (\delta p)v^*$, which gives $(\delta A)v = \delta p$ with $\|\delta A\| = \|\delta p\|$ (Example 3.6). Now it is clear that to obtain the maximum tilt with a given $\|\delta p\|$, we should take p to be as close to the origin as possible. That is, we want $p = \sigma_n u_n$, where σ_n is the smallest singular value of A and u_n is the corresponding left singular vector. With A in the diagonal form (18.7), p is equal to the last column of A, v^* is the n-vector $(0, 0, \ldots, 0, 1)$, and δA is a perturbation of the entries of A below the diagonal in this column. Such a perturbation tilts range(A) by the angle $\delta\alpha$ given by $\tan(\delta\alpha) = \|\delta p\|/\sigma_n$. Since $\|\delta p\| = \|\delta A\|$ and $\delta\alpha \le \tan(\delta\alpha)$, we have

$$\delta\alpha \le \frac{\|\delta A\|}{\sigma_n} = \frac{\|\delta A\|}{\|A\|}\kappa(A), \tag{18.10}$$

with equality attained for choices δA of the kind just described, provided they are infinitesimal (so that $\delta\alpha = \tan(\delta\alpha)$).

Sensitivity of y to Perturbations in A

Now we are prepared to derive the second row of the table in Theorem 18.1. We begin with its left-hand entry. Since y is the orthogonal projection of b onto range(A), it is determined by b and range(A) alone. Therefore, to analyze the sensitivity of y to perturbations in A, we can simply study the effect on y of tilting range(A) by some angle $\delta\alpha$.

An elegant geometrical property becomes apparent when we imagine fixing b and watching y vary as range(A) is tilted (Figure 18.2). No matter how range(A) is tilted, the vector $y \in$ range(A) must always be orthogonal to $y - b$. That is, the line b–y must lie at right angles to the line 0–y. In other words, as range(A) is adjusted, y moves along the sphere of radius $\|b\|/2$ centered at the point $b/2$.

Tilting range(A) in the plane 0–b–y by an angle $\delta\alpha$ changes the angle 2θ at the central point $b/2$ by $2\delta\alpha$. Thus the corresponding perturbation δy is the

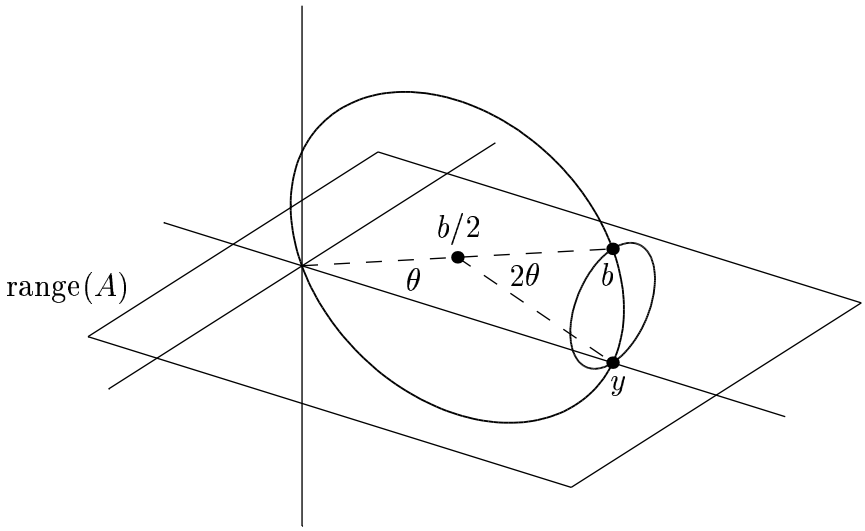


Figure 18.2. *Two circles on the sphere along which y moves as* range(A) *varies. The large circle, of radius $\|b\|/2$, corresponds to tilting* range(A) *in the plane 0–b–y, and the small circle, of radius $(\|b\|/2)\sin\theta$, corresponds to tilting it in an orthogonal direction. However* range(A) *is tilted, y remains on the sphere of radius $\|b\|/2$ centered at $b/2$.*

base of an isosceles triangle with central angle $2\delta\alpha$ and edge length $\|b\|/2$. This implies $\|\delta y\| = \|b\|\sin(\delta\alpha)$. Tilting range$(A)$ in any other direction results in a similar geometry in a different plane and perturbations smaller by a factor as small as $\sin\theta$. Thus for arbitrary perturbations by an angle $\delta\alpha$ we have

$$\|\delta y\| \;\le\; \|b\|\sin(\delta\alpha) \;\le\; \|b\|\delta\alpha. \tag{18.11}$$

By (18.4) and (18.10), this gives us $\|\delta y\| \le \|\delta A\|\kappa(A)\|y\|/\|A\|\cos\theta$, that is,

$$\frac{\|\delta y\|}{\|y\|} \Big/ \frac{\|\delta A\|}{\|A\|} \;\le\; \frac{\kappa(A)}{\cos\theta}. \tag{18.12}$$

This establishes the lower-left result of Theorem 18.1.

Sensitivity of x to Perturbations in A

We are now ready to analyze the most interesting relationship of Theorem 18.1: the sensitivity of x to perturbations in A.

A perturbation δA splits naturally into two parts: one part δA_1 in the first n rows of A, and another part δA_2 in the remaining $m-n$ rows:

$$\delta A \;=\; \begin{bmatrix} \delta A_1 \\ \delta A_2 \end{bmatrix} \;=\; \begin{bmatrix} \delta A_1 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ \delta A_2 \end{bmatrix}.$$

First, let us consider the effect of perturbations δA_1. Such a perturbation changes the mapping of A in its range, but not range(A) itself or y. It perturbs A_1 by δA_1 in the square system (18.8) without changing b_1. The condition number for such perturbations is given by (12.18), which here takes the form

$$\frac{\|\delta x\|}{\|x\|} \Big/ \frac{\|\delta A_1\|}{\|A\|} \;\le\; \kappa(A_1) \;=\; \kappa(A). \tag{18.13}$$

Next we consider the effect of (infinitesimal) perturbations δA_2. Such a perturbation tilts range(A) without changing the mapping of A within this space. The point y and thus the vector b_1 are perturbed, but not A_1. This corresponds to perturbing b_1 in (18.8) without changing A_1. The condition number for such perturbations is given by (12.14), which takes the form

$$\frac{\|\delta x\|}{\|x\|} \Big/ \frac{\|\delta b_1\|}{\|b_1\|} \;\le\; \frac{\kappa(A_1)}{\eta(A_1;x)} \;=\; \frac{\kappa(A)}{\eta}. \tag{18.14}$$

To finish the argument we need to relate δb_1 to δA_2. Now the vector b_1 is y expressed in the coordinates of range(A). Therefore, the only changes in y that are realized as changes in b_1 are those that lie parallel to range(A); orthogonal changes have no effect. In particular, if range(A) is tilted by an angle $\delta\alpha$ in the plane 0–b–y, the resulting perturbation δy lies not parallel to range(A) but at an angle $\pi/2 - \theta$. Consequently, the change in b_1 satisfies $\|\delta b_1\| = \sin\theta \|\delta y\|$. By (18.11), we therefore have

$$\|\delta b_1\| \le (\|b\|\delta\alpha)\sin\theta. \tag{18.15}$$

Curiously, if range(A) is tilted in a direction orthogonal to the plane 0–b–y, we obtain the same bound, but for a different reason. Now δy is parallel to range(A), but it is a factor of $\sin\theta$ smaller, as discussed above in connection with Figure 18.2. Thus we have $\|\delta y\| \le (\|b\|\delta\alpha)\sin\theta$, and since $\|\delta b_1\| \le \|\delta y\|$, we again arrive at (18.15).

All the pieces are now in place. Since $\|b_1\| = \|b\|\cos\theta$, we can rewrite (18.15) as

$$\frac{\|\delta b_1\|}{\|b_1\|} \le (\delta\alpha)\tan\theta. \tag{18.16}$$

Relating $\delta\alpha$ to $\|\delta A_2\|$ by (18.10) and combining (18.14) with (18.16), we obtain

$$\frac{\|\delta x\|}{\|x\|} \Big/ \frac{\|\delta A_2\|}{\|A\|} \;\le\; \frac{\kappa(A)^2\tan\theta}{\eta}.$$

Adding this to (18.13) establishes the lower-right result of Theorem 18.1.

Exercises

18.1. Consider the example

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 1.0001 \\ 1 & 1.0001 \end{bmatrix}, \qquad b = \begin{bmatrix} 2 \\ 0.0001 \\ 4.0001 \end{bmatrix}.$$

(a) What are the matrices A^+ and P for this example? Give exact answers.

(b) Find the exact solutions x and $y = Ax$ to the least squares problem $Ax \approx b$.

(c) What are $\kappa(A)$, θ, and η? From here on, numerical answers are acceptable.

(d) What are the four condition numbers of Theorem 18.1?

(e) Give examples of perturbations δb and δA that approximately attain these four condition numbers.

18.2. Social scientists depend on the technique of *regression*, in which a vector of observations of some quantity is approximated in the least squares sense by a linear combination of other vectors. The coefficients of the fit are then interpreted as representing, say, the effects on annual income of IQ, years of education, parents' years of education, and parents' income.

One might think that the more variables one included in such a model, the more information one would obtain, but this is not always true. Explain this phenomenon from the point of view of conditioning, making specific reference to the results of Theorem 18.1.

18.3. Suppose you look across Cayuga Lake at a light from a house on the other side. If the lake surface is rippled, the reflected light appears as a long vertical streak. The same effect appears with taillights of the car ahead of you on a rainy road, or even with reflections of hallway lights on a shiny waxed floor. It is a real effect, not an optical illusion, and the explanation is a matter of geometry.

(a) Derive a quantitative theory explaining this phenomenon. Specifically, suppose you and the house across the lake are each fifty meters above the surface, and the lake is one kilometer wide. What is the length-to-width ratio of the streak as it appears in your visual field?

(b) Describe a connection between this problem and one of the geometrical arguments of this lecture.

18.4. Explain why, as remarked after Theorem 18.1, the condition number of y with respect to perturbations in A becomes 0 in the case $m = n$.

Lecture 19. Stability of Least Squares Algorithms

Least squares problems can be solved by various methods, as described in Lecture 11, including the normal equations, Householder triangularization, Gram–Schmidt orthogonalization, and the SVD. Here we compare these methods and show that the use of the normal equations is in general unstable.

Example

To illustrate the behavior of our algorithms, we shall apply them to a numerical example with $m = 100$, $n = 15$. Here is the MATLAB setup:

```
m = 100; n = 15;
t = (0:m-1)'/(m-1);          Set t to a discretization of [0,1].
A = []; for i=1:n,           Construct Vandermonde matrix.
  A = [A t.^(i-1)]; end
b = exp(sin(4*t));           Right-hand side.
b = b/2006.787453080206;     Normalization (see text).
```

The idea behind this example is the least squares fitting of the function $\exp(\sin(4\tau))$ on the interval $[0,1]$ by a polynomial of degree 14. First we discretize $[0,1]$, defining a vector t of 100 points equally spaced from 0 to 1. The matrix A is the 100×15 Vandermonde matrix whose columns are the

powers $1, \tau, \ldots, \tau^{14}$ sampled at the points of t, and the right-hand side b is the function $\exp(\sin(4\tau))$ sampled at these points.

The reason for the bizarre final line of the code is as follows. For simplicity, we are going to compare just the coefficients x_{15} computed by our various algorithms. Without this final line, the correct value of x_{15} would be 2006.787453080206... (this figure was obtained with an extended precision arithmetic package). By dividing by this number, we obtain a problem whose solution has $x_{15} = 1$, making our comparisons easier to follow.

To explain our observations, we shall need the quantities (18.3)–(18.5). One can determine these to sufficient accuracy by solving the least squares problem numerically with the aid of MATLAB's \ operator:

```
x = A\b; y = A*x;                    Solve least squares problem.
kappa = cond(A)
    kappa = 2.2718e+10                   κ(A)
theta = asin(norm(b-y)/norm(b))
    theta = 3.7461e-06                   θ
eta = norm(A)*norm(x)/norm(y)
    eta = 2.1036e+05                     η
```

The result $\kappa(A) \approx 10^{10}$ indicates that the monomials $1, t, \ldots, t^{14}$ form a highly ill-conditioned basis. The result $\theta \approx 10^{-6}$ indicates that $\exp(\sin(4t))$ can be fitted very closely by a polynomial of degree 14. (The fit is so close that we computed θ with the formula $\theta = \sin^{-1}(\|b-y\|/\|b\|)$ instead of (18.4), to avoid cancellation error.) As for η, its value of about 10^5 is about midway between the extremes 1 and $\kappa(A)$ permitted by (18.6).

Inserting these numbers into the formulas of Theorem 18.1, we find that for our example problem, the condition numbers of y and x with respect to perturbations in b and A are approximately

	y	x
b	1.0	1.1×10^5
A	2.3×10^{10}	3.2×10^{10}

Householder Triangularization

As mentioned in Lecture 11, the standard algorithm for solving least squares problems is QR factorization via Householder triangularization (Algorithm 11.2). Here is what we get with a MATLAB experiment:

```
[Q,R] = qr(A,0);                     Householder triang. of A.
x = R\(Q'*b);                        Solve for x.
x(15)
   ans = 1.00000031528723
```

What can we make of this result? Thanks to our normalization, the correct answer would be $x_{15} = 1$. Thus we have a relative error of about 3×10^{-7}. Since the calculation was done in IEEE double precision arithmetic with $\epsilon_{\text{machine}} \approx 10^{-16}$, this means that the rounding errors have been amplified by a factor of order 10^9. At first sight this looks bad, but a glance at the table above reminds us that the condition number of x with respect to perturbations in A is of order 10^{10}. Thus the inaccuracy in x_{15} can be entirely explained by ill-conditioning, not instability. Algorithm 11.2 appears to be backward stable.

Above, we formed $\hat{Q}$ explicitly, but as emphasized in Lectures 10 and 16, this is not necessary. It is enough to store the vectors v_k determined at the kth step of Algorithm 10.1 (equation (10.5)), which can then be utilized to compute $\hat{Q}^* b$ by Algorithm 10.2. In MATLAB, we can achieve this effect by computing a QR factorization not just of A but of the $m \times (n+1)$ "augmented" matrix $[A \;\; b]$. In the course of this factorization, the n Householder reflectors that make A upper-triangular are applied to b also, leaving the vector $\hat{Q}^* b$ in the first n positions of column $n+1$. An additional $(n+1)$st reflector is then applied to make entries $n+2, \ldots, m$ of column $n+1$ zero, but this does not change the first n entries of that column, which are the ones we care about. Thus:

```
[Q,R] = qr([A b],0);          Householder triang. of [A b].
Qb = R(1:n,n+1);              Extract Q̂*b ...
R = R(1:n,1:n);                  ... and R̂.
x = R\Qb;                     Solve for x.
x(15)
   ans = 1.00000031529465
```

The answer is almost the same as before. This indicates that the errors introduced in the QR factorization of A swamp those introduced in the computation of $\hat{Q}^* b$.

There is also a third way to solve the least squares problem via Householder triangularization in MATLAB. We can use the built-in operator \, as we did already in finding $\kappa(A)$, θ, and η:

```
x = A\b;                      Solve for x.
x(15)
   ans = 0.99999994311087
```

This result is distinctly different from the others, and an order of magnitude more accurate. The reason for this is that MATLAB's \ operator makes use of *QR factorization with column pivoting*, based on a factorization $AP = \hat{Q}\hat{R}$, where P is a permutation matrix. In this book we shall not discuss column pivoting.

From the point of view of normwise stability analysis, these three variants of QR factorization are equal. All of them, it can be proved, are backward stable.

Theorem 19.1. *Let the full-rank least squares problem* (11.2) *be solved by Householder triangularization (Algorithm* 11.2*) on a computer satisfying* (13.5) *and* (13.7). *This algorithm is backward stable in the sense that the computed solution* $\tilde{x}$ *has the property*

$$\|(A+\delta A)\tilde{x}-b\| = \min, \qquad \frac{\|\delta A\|}{\|A\|} = O(\epsilon_{\text{machine}}) \tag{19.1}$$

for some $\delta A \in \mathbb{C}^{m\times n}$. *This is true whether* $\hat{Q}^*b$ *is computed via explicit formation of* $\hat{Q}$ *or implicitly by Algorithm* 10.2. *It also holds for Householder triangularization with arbitrary column pivoting.*

Gram–Schmidt Orthogonalization

Another way to solve a least squares problem is by modified Gram–Schmidt orthogonalization (Algorithm 8.1). For $m \approx n$, this takes somewhat more operations than the Householder approach, but for $m \gg n$, the flop counts for both algorithms are asymptotic to $2mn^2$.

The following MATLAB sequence implements this algorithm in the obvious fashion. The function `mgs` is an implementation (not shown) of Algorithm 8.1—the same as in Experiment 2 of Lecture 9.

```
[Q,R] = mgs(A);                    Gram–Schmidt orthog. of A.
x = R\(Q'*b);                      Solve for x.
x(15)
   ans = 1.02926594532672
```

This result is very poor. Rounding errors have been amplified by a factor on the order of 10^{14}, far greater than the condition number of the problem. In fact, this algorithm is unstable, and the reason is easily identified. As mentioned at the end of Lecture 9, Gram–Schmidt orthogonalization produces matrices $\hat{Q}$, in general, whose columns are not accurately orthonormal. Since the algorithm above depends on that orthonormality, it suffers accordingly.

The instability can be avoided by a reformulation of the algorithm. Since the Gram–Schmidt iteration delivers an accurate product $\hat{Q}\hat{R}$, even if $\hat{Q}$ does not have accurately orthogonal columns, one approach is to set up the normal equations $Rx = (\hat{Q}^*\hat{Q})^{-1}\hat{Q}^*b$ for the vector Rx, then get x by back substitution. As long as the computed $\hat{Q}$ is at least well-conditioned, this method will be free of the instabilities described below for the normal equations applied to arbitrary matrices. However, it involves unnecessary extra work and should not be used in practice.

A better method of stabilizing the Gram–Schmidt method is to make use of an augmented system of equations, just as in the second of our two Householder experiments above:

```
[Q,R] = mgs([A b]);          Gram–Schmidt orthog. of [A  b].
Qb = R(1:n,n+1);             Extract Q̂*b ...
R = R(1:n,1:n);                 ...and R̂.
x = R\Qb;                    Solve for x.
x(15)
   ans = 1.00000005653399
```

Now the result looks as good as with Householder triangularization. It can be proved that this is always the case.

Theorem 19.2. *The solution of the full-rank least squares problem* (11.2) *by Gram–Schmidt orthogonalization is also backward stable, satisfying* (19.1), *provided that $\hat{Q}^*b$ is formed implicitly as indicated in the code segment above.*

Normal Equations

A fundamentally different approach to least squares problems is the solution of the normal equations (Algorithm 11.1), typically by Cholesky factorization (Lecture 23). For $m \gg n$, this method is twice as fast as methods depending on explicit orthogonalization, requiring asymptotically only mn^2 flops (11.14). In the following experiment, the problem is solved in a single line of MATLAB by the \ operator:

```
x = (A'*A)\(A'*b);           Form and solve normal equations.
x(15)
   ans = 0.39339069870283
```

This result is terrible! It is the worst we have obtained, with not even a single digit of accuracy. The use of the normal equations is clearly an unstable method for solving least squares problems. We shall take a moment to explain this phenomenon, for the explanation is a perfect example of the interplay of ideas of conditioning and stability. Also, the normal equations are so often used that an understanding of the risks involved is important.

Suppose we have a backward stable algorithm for the full-rank problem (11.2) that delivers a solution $\tilde{x}$ satisfying $\|(A+\delta A)\tilde{x} - b\| = \min$ for some δA with $\|\delta A\|/\|A\| = O(\epsilon_{\text{machine}})$. (Allowing perturbations in b as well as A, or considering stability instead of backward stability, does not change our main points.) By Theorems 15.1 and 18.1, we have

$$\frac{\|\tilde{x} - x\|}{\|x\|} \;=\; O\left(\left(\kappa + \frac{\kappa^2 \tan\theta}{\eta}\right)\epsilon_{\text{machine}}\right), \tag{19.2}$$

where $\kappa = \kappa(A)$. Now suppose A is ill-conditioned, i.e., $\kappa \gg 1$, and θ is bounded away from $\pi/2$. Depending on the values of the various parameters, two very different situations may arise. If $\tan\theta$ is of order 1 (that is, the least squares fit is not especially close) and $\eta \ll \kappa$, the right-hand side (19.2) is $O(\kappa^2\epsilon_{\text{machine}})$. On the other hand, if $\tan\theta$ is close to zero (a very close fit) or η is close to κ, the bound is $O(\kappa\epsilon_{\text{machine}})$. *The condition number of the least squares problem may lie anywhere in the range κ to κ^2.*

Now consider what happens when we solve (11.2) by the normal equations, $(A^*A)x = A^*b$. Cholesky factorization is a stable algorithm for this system of equations in the sense that it produces a solution $\tilde{x}$ satisfying $(A^*A + \delta H)\tilde{x} = A^*b$ for some δH with $\|\delta H\|/\|A^*A\| = O(\epsilon_{\text{machine}})$ (Theorem 23.3). However, the matrix A^*A has condition number κ^2, not κ. Thus the best we can expect from the normal equations is

$$\frac{\|\tilde{x} - x\|}{\|x\|} \;=\; O(\kappa^2\epsilon_{\text{machine}}). \tag{19.3}$$

The behavior of the normal equations is governed by κ^2, not κ.

The conclusion is now clear. If $\tan\theta$ is of order 1 and $\eta \ll \kappa$, or if κ is of order 1, then (19.2) and (19.3) are of the same order and the normal equations are stable. If κ is large and either $\tan\theta$ is close to zero or η is close to κ, however, then (19.3) is much bigger than (19.2) and the normal equations are unstable. *The normal equations are typically unstable for ill-conditioned problems involving close fits.* In our example problem, with $\kappa^2 \approx 10^{20}$, it is hardly surprising that Cholesky factorization yielded no correct digits.

According to our definitions, an algorithm is stable only if it has satisfactory behavior uniformly across all the problems under consideration. The following result is thus a natural formalization of the observations just made.

Theorem 19.3. *The solution of the full-rank least squares problem* (11.2) *via the normal equations (Algorithm* 11.1*) is unstable. Stability can be achieved, however, by restriction to a class of problems in which $\kappa(A)$ is uniformly bounded above or $(\tan\theta)/\eta$ is uniformly bounded below.*

SVD

One further algorithm for least squares problems was mentioned in Lecture 11: the use of the SVD (Algorithm 11.3). Like most computations based on the SVD, this one is stable:

```
[U,S,V] = svd(A,0);          Reduced SVD of A.
x = V*(S\(U'*b));            Solve for x.
x(15)
   ans = 0.99999998230471
```

In fact, this is the most accurate of all the results obtained in our experiments, beating Householder triangularization with column pivoting (MATLAB's \ ) by a factor of about 3. A theorem in the usual form can be proved.

Theorem 19.4. *The solution of the full-rank least squares problem* (11.2) *by the SVD (Algorithm* 11.3*) is backward stable, satisfying the estimate* (19.1).

Rank-Deficient Least Squares Problems

In this lecture we have identified four backward stable algorithms for linear least squares problems: Householder triangularization, Householder triangularization with column pivoting, modified Gram–Schmidt with implicit calculation of $\hat{Q}^*b$, and the SVD. From the point of view of classical normwise stability analysis of the full-rank problem (11.2), the differences among these algorithms are minor, so one might as well make use of the simplest and cheapest, Householder triangularization without pivoting.

However, there are other kinds of least squares problems where column pivoting and the SVD take on a special importance. These are problems where A has rank $< n$, possibly with $m < n$, so that the system of equations is *underdetermined*. Such problems do not have a unique solution unless one adds an additional condition, typically that x itself should have as small a norm as possible. A further complication is that the correct solution depends on the rank of A, and determining ranks numerically in the presence of rounding errors is never a trivial matter.

Thus rank-deficient least squares problems are not a challenging subclass of least squares problems, but fundamentally different. Since the definition of a solution is new, there is no reason that an algorithm that is stable for full-rank problems must be stable also in the rank-deficient case. In fact, the only fully stable algorithms for rank-deficient problems are those based on the SVD. An alternative is Householder triangularization with column pivoting, which is stable for almost all problems. We shall not give details.

Exercises

19.1. Given $A \in \mathbb{C}^{m\times n}$ of rank n and $b \in \mathbb{C}^m$, consider the block 2×2 system of equations

$$\begin{bmatrix} I & A \\ A^* & 0 \end{bmatrix} \begin{bmatrix} r \\ x \end{bmatrix} = \begin{bmatrix} b \\ 0 \end{bmatrix}, \tag{19.4}$$

where I is the $m \times m$ identity. Show that this system has a unique solution $(r, x)^T$, and that the vectors r and x are the residual and the solution of the least squares problem (18.1).

19.2. Here is a stripped-down version of one of MATLAB's built-in m-files.

```
[U,S,V] = svd(A);
S = diag(S);
tol = max(size(A))*S(1)*eps;
r = sum(S > tol);
S = diag(ones(r,1)./S(1:r));
X = V(:,1:r)*S*U(:,1:r)';
```

What does this program compute?